Artifact Evaluated
NDSS
Available
Functional
Reproduced

OrionLink: Single Sign-On with Oblivious Identity Brokers

Yijia Fang*‡, Bingyu Li*✉, Luchao Jin*, Jingqiang Lin†,
Mingxun Zhou‡, Zhijintong Zhang*, Linghui Li§, Qianhong Wu*
*School of Cyber Science and Technology, Beihang University, China
†School of Cyber Science and Technology, University of Science and Technology of China, China
‡The Hong Kong University of Science and Technology
§Beijing University of Posts and Telecommunications
{fangyj, libingyu, jinluchao, zhangzjt, qianhong.wu}@buaa.edu.cn
linjq@ustc.edu.cn, mingxunz@ust.hk, lilinghui@bupt.edu.cn

***Abstract*—Single Sign-On (SSO) is a federated authentication mechanism that improves user experience by enabling access to multiple services, or Relying Parties (RPs), through a single authentication at an Identity Provider (IdP). To reduce integration complexity, many RPs rely on third-party *Identity Brokers (Brokers)* that aggregate numerous IdPs into a unified interface, thereby simplifying protocol management and interoperability. However, while brokers ease development, they introduce a fundamental security and privacy risk: sensitive authentication credentials and user behavioral data are exposed to the broker, creating a high-value single point of failure. Existing SSO solutions provide limited protection against these risks.**

With the goal of achieving broker obliviousness, we address these challenges at their root and present *OrionLink*, a comprehensive privacy-enhancing SSO pipeline that is compatible with existing brokered SSO architectures and systematically mitigates broker-induced risks without sacrificing usability. OrionLink incorporates two core components: (*a*) an *anonymous pseudonym derivation scheme* that generates unlinkable, per-RP identifiers to prevent cross-service user tracking; (*b*) an *anonymous key exchange protocol* that establishes end-to-end encrypted channels to protect user data from broker visibility. We further instantiate OrionLink with two new features, preserving broker functionality and deployability. We implemented a prototype of OrionLink within a brokered SSO environment, and experimental evaluation shows that the framework achieves substantial security and privacy improvements with acceptable performance overhead.

***Index Terms*—Single Sign-On, Authentication, End-to-End Encryption, Privacy**

I. Introduction

Single Sign-On (SSO) is one of the most widely adopted federated authentication mechanisms, commonly realized through standards such as OpenID Connect 1.0 (OIDC) [1] and OAuth 2.0 [2]. It allows users to access websites and services, known as relying parties (RPs), with a pre-registered account managed by an identity provider (IdP), significantly reducing registration and account management burdens. The adoption of SSO is widespread; more than 45,000 websites in the Tranco Top 1M [3] rely on major IdPs such as Apple, Google, Microsoft, and Facebook [4]. Over the past decade, numerous studies have analyzed the security of SSO [4]–[10], and the IETF regularly revises its specifications [1], [11], such as the *Security Best Current Practices (SBCPs)* [12], to incorporate these findings.

Brokered SSO. Brokered SSO is an increasingly adopted architecture in which an intermediate *identity broker* (IdB) mediates interactions between relying parties (RPs) and identity providers (IdPs). The broker exposes a unified authentication interface to RPs while internally managing the heterogeneous, protocol-specific authentication workflows of multiple IdPs. This design relieves RPs from the need to directly integrate with diverse identity systems. Figure 1 contrasts conventional three-party SSO with brokered SSO.

The primary advantage of brokered SSO is reduced deployment and maintenance overhead. By integrating with a single broker, an RP can simultaneously support authentication from many IdPs, analogous to payment aggregators (e.g., Stripe or Alipay) abstracting bank-specific protocols for online merchants.

Adoption of brokered SSO has grown rapidly. A 2025 measurement study [13] reports that approximately 25% of SSO-enabled websites rely on identity brokers. Emerging standards, such as the Model Context Protocol (MCP) [14], further increase demand for brokered authentication [15], [16]. As a result, identity brokers have become a critical component of the authentication ecosystem, with significant security and privacy implications for both RPs and end users (see Appendix B-A).

Data Exposure Risk in Brokered SSO. While identity brokers simplify SSO deployment, they introduce a fundamental security and privacy risk by concentrating sensitive authentication and behavioral data at a single intermediary, thereby creating a high-value point of failure. At a minimum, brokers can observe users' cross-site authentication activities, enabling fine-grained tracking and potential secondary use of privacy-sensitive information.

More critically, brokers routinely process high-impact authentication data across many RPs, including plaintext user identifiers, access tokens, and session cookies. This aggre-

✉ Bingyu Li is the corresponding author.

Network and Distributed System Security (NDSS) Symposium 2027
22–26 March 2027, Seoul, Republic of Korea
ISBN 978-1-970672-09-1
https://dx.doi.org/10.14722/ndss.2027.231871
www.ndss-symposium.org

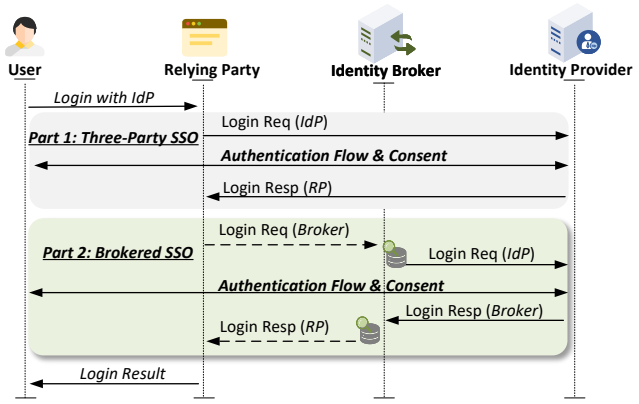
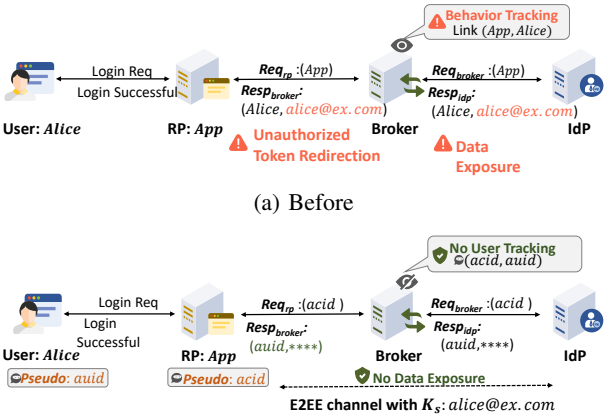


Fig. 1: Traditional three-party SSO (top) features direct RP–IdP interaction, while brokered SSO (bottom) inserts an identity broker that relays and re-issues authentication tokens.



(a) Before

(b) OrionLink

Fig. 2: Improving Privacy: Before vs. OrionLink.

gation makes brokers attractive targets for attackers, with potentially severe downstream consequences. In 2023, for example, a major identity broker, *Okta*, was compromised via its customer support systems, exposing HTTP Archive (HAR) files that contained authentication tokens and session cookies for multiple RPs [17]. The incident affected several foundational infrastructure providers, including Cloudflare [16] and 1Password [18], and propagated to their dependent services.

Beyond isolated breaches, systemic risks are also evident. A large-scale study by Innocenti et al. [13] found that 49 brokers inadequately validate redirection chains, potentially exposing user data or authentication tokens to attackers. In some cases, these flaws enable full account takeover through token leakage.

Given the current state of brokered SSO deployments, we pose the following fundamental research question:

> *Is it possible to design a brokered SSO system that minimizes data leakage at the broker while still providing a unified authentication interface with practical efficiency?*

Our Contribution. We take a more fundamental approach to addressing the security and privacy risks of brokered SSO. Rather than introducing additional protection mechanisms to secure data already exposed to identity brokers, we aim to *minimize the information observable by the broker in the first place*. To this end, we design a new brokered SSO pipeline, **OrionLink**, based on cryptographic engineering approaches.

The central design goal of OrionLink is *broker obliviousness*. In OrionLink, the broker does not learn users' real identities and acts solely as a forwarding intermediary between RPs and IdPs. Sensitive authentication data are protected using end-to-end encryption (E2EE), ensuring that even a compromised broker cannot access critical private information or exfiltrate it retrospectively (as shown in Figure 2).

Concretely, OrionLink enforces multiple forms of broker obliviousness through carefully tailored cryptographic designs (summarized in Table I), including:

1) *Identity Obliviousness: The broker does not learn the user's real identity in any SSO session.* In each session, the RP derives an anonymous identifier and generates a corresponding authenticity proof that is verified by the IdP. The IdP then applies an Oblivious Pseudo-Random Function (OPRF) [19] to blindly derive a pairwise pseudonym for the user, bound to the RP's anonymous identifier. This construction ensures that pseudonyms are unique per RP, preventing cross-domain tracking while preserving the session consistency required for broker-side data handling.
2) *Authorization Credential Obliviousness: The broker does not learn authorization credentials (e.g., access tokens) used to access user resources at the IdP.* In each session, the IdP anonymously authenticates the RP and verifies that it is registered and authorized by the user. We further design an *anonymous authentication key agreement (AAKA)* protocol that enables the IdP and RP to establish a secure session key. This key is used to encrypt authorization credentials in the login response, which is forwarded by the broker and can only be decrypted by the authorized RP within the active session. As a result, the broker never observes authorization credentials in plaintext, mitigating the risk of credential leakage and unauthorized resource access.
3) *Associated Personal Information Obliviousness: The broker does not learn the user's personal information (e.g., email address, name, or phone number).* Similar to authorization credentials, personal information is encrypted by the IdP in the login response and forwarded through the broker such that only the authorized RP can decrypt it. Since the broker neither possesses authorization credentials nor access to plaintext user attributes, it cannot harvest personal information or query user resources at the IdP, thereby substantially reducing data leakage risks.

Throughout our design, we assume an *honest-but-curious* identity broker: the broker correctly follows the protocol specification but attempts to infer sensitive information from all data it can observe. This threat model captures a broad range of realistic risks, including data breaches, as any information visible to the broker is assumed to be ultimately accessible to an adversary. Our model therefore reflects the practical

TABLE I: Broker Information Exposure: Before vs. OrionLink.

Information Type	Before	OrionLink
Unique user identifier	⚠	◐[1]
Cross-RP linkability	⚠	◐
User attributes (e.g., email, name, phone)	⚠	◐
Access token	⚠	◐
Refresh token	⚠	◐[2]

[1] The broker observes only an RP-specific pseudonym rather than the user's real identifier.
[2] The broker receives a refresh token but cannot access the refreshed access token or its contents.

risk that broker-observed information may be exposed through channels outside the primary authentication logic. Designing brokered SSO protocols secure against a fully malicious broker is an important direction for future work and lies beyond the scope of this paper.

In addition, we instantiate OrionLink on top of the widely deployed OIDC authorization code flow [1] and introduce two new features to improve practical usability: (*a*) *private authorization management* and (*b*) *forward-secure token refresh*.

The first feature enables users to review and revoke RP accesses without revealing authorization relationships to the IdP. This addresses a critical yet often overlooked gap in prior privacy-preserving SSO systems [20]–[23]: although anonymous IdP-to-RP authentication prevents the IdP from tracking users' authorization choices, users still require visibility and control over their authorized RPs for tasks such as access review and revocation. The second feature allows the broker to assist RPs in securely maintaining token validity using refresh tokens. Each refresh response is encrypted under a freshly derived session key, providing forward secrecy and post-compromise security.

OrionLink is usable and efficient in practice, and it remains largely transparent to the user. Our evaluation shows that OrionLink incurs an end-to-end login latency of 2.08 seconds in a WAN setting, with additional local computation overhead below 0.07 seconds. Compared to a standard brokered SSO pipeline, which typically requires approximately 0.5 seconds per login, OrionLink introduces only modest overhead while providing substantially stronger security and privacy guarantees. These results indicate that OrionLink meets real-world performance requirements and is suitable for practical deployment.

In summary, we make the following contributions:

- **Problem formulation.** We identify *data exposure at identity brokers* as a fundamental security and privacy risk in brokered SSO systems and formalize the brokered SSO pipeline through a rigorous system model.
- **System design.** We design OrionLink, a new brokered SSO pipeline that enforces *broker obliviousness* through cryptographic constructions, ensuring that brokers neither learn user identities nor access sensitive authentication data, thereby preventing data leakage in a principled manner. We further instantiate OrionLink on OIDC and introduce two usability-enhancing features.
- **Formal security analysis.** We formally define the security and privacy goals of broker obliviousness and provide rigorous cryptographic proofs.
- **Implementation and evaluation.** We implement and open-source a prototype of OrionLink and evaluate its performance and usability, demonstrating that it achieves strong security guarantees with only modest overhead compared to standard brokered SSO pipelines, making it suitable for real-world deployment.

II. Preliminaries and Building Blocks

In this section, we outline the preliminaries and foundational blocks necessary for OrionLink.

Notations: Let $\mathbb{G}$ be a cyclic group of prime order p with generator g, and denote $\mathbb{G}^* = \mathbb{G} \setminus \{1_{\mathbb{G}}\}$. All group operations are written multiplicatively, with scalar exponentiation denoted as X^a for $X \in \mathbb{G}$ and $a \in \mathbb{Z}_p$.

A. Brokered SSO Flow

Brokered SSO enables RPs to connect to IdPs through a single broker, which provides a range of functionalities to reduce deployment and maintenance overhead, including:

1) *Unified Authentication Interface.* Exposes a single standardized authentication interface, allowing applications to support heterogeneous IdPs without IdP-specific logic.
2) *Identity Normalization.* Maps identities from diverse IdPs into a unified user representation, enabling RPs to operate on stable and standardized claims.
3) *Centralized Security Enforcement.* Centralizes security-critical mechanisms, including token validation, token refresh, MFA, and risk control.
4) *Scalability.* Allows IdPs to be added or updated at the broker without requiring RP-side changes.

Figure 3 shows the standard brokered SSO pipeline, consisting of a *Registration* phase and a *Login Flow* phase. Overall, the broker performs two standard SSO flows independently: first acting as a proxy RP to authenticate with the IdP, and then as a proxy IdP to authorize the RP.

Phase 1: Registration. ① The RP registers with the IdP and obtains `client_idb` (i.e., `client_id` and `client_secret`). ② The RP forwards `client_idb` to the broker to register and receives `client_rp`.

Phase 2: Login Flow. ①–② When the user selects *"Login with IdP"* on the RP, the RP redirects the authentication request to the broker. ③ The broker generates a standardized request according to the specific IdP and forwards it to the IdP to initiate authentication. ④ The user verifies the RP's identity on the frontend and decides whether to grant authorization. ⑤ After authenticating the user and obtaining consent, the IdP returns a token containing the user's personal information (`userinfo`) to the broker. In the OIDC Authentication Code flow, the broker additionally exchanges the authorization code for an `id_token` or `access_token`. ⑥ The broker verifies the token signature, parses its content, and records the RP–user mapping. Then the broker issues a new token for the RP and

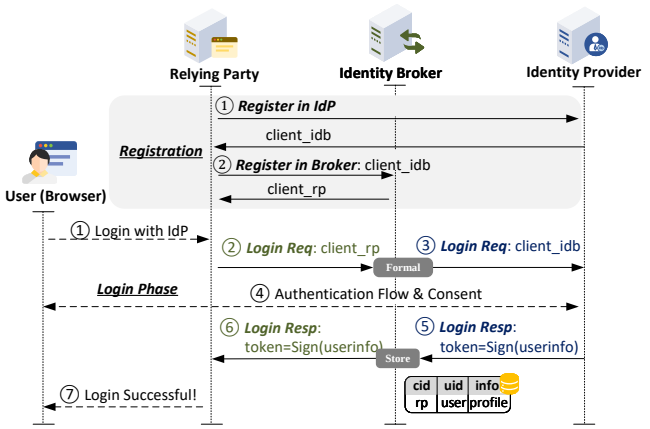


Fig. 3: Brokered SSO Login Flow.

redirects the browser back to the RP. ⑦ The RP validates the token, establishes a local session, and grants the user access.

B. Oblivious Pseudo-Random Functions

An OPRF [19], [24] is a two-party protocol where the server holds a PRF key k, and the client provides an input X. The client learns only the output $\mathsf{OPRF}_k(X)$, while the server learns nothing about X or the final output.

We adopt OPRFs over prime-order groups [25], [26]:

- $(r, \alpha) \leftarrow \mathsf{Blind}(x)$: The client samples a random blinding factor $r \xleftarrow{\$} \mathbb{Z}_p^*$, computes the blinded element $\alpha = H(x)^r$, and outputs (r, α), where $H : \{0,1\}^* \to \mathbb{G}$ is a hash-to-group function.
- $\beta \leftarrow \mathsf{BlindEval}(k, \alpha)$: The server computes $\beta = \alpha^k$ using its secret key $k \in \mathbb{Z}_p^*$ and returns β.
- $y \leftarrow \mathsf{Unblind}(r, \beta)$: The client computes $y = \beta^{1/r} = H(x)^k$ to remove the blinding from the input value β, and outputs unblinded element y. In practice, y is often hashed together with the input x to produce the final OPRF output.

This protocol satisfies *correctness*, *consistency*, and *pseudorandomness* for which we provide formal definitions in Appendix G-A.

C. Proof of Knowledge

A Proof of Knowledge (PoK) allows a prover to convince a verifier of knowledge of a secret witness without revealing it. We use non-interactive PoKs derived from Σ-protocols [27] via the Fiat–Shamir transform [28]. A non-interactive zero-knowledge PoK is represented as:

$$\mathsf{PoK}\{(x_1, x_2, ...) : \text{statements about } x_1, x_2, ...\},$$

where $(x_1, x_2, ...)$ are secret values that, together with public parameters, satisfy the specified statement.

D. Anonymous Credentials

Anonymous credentials [29] allow a *prover* to obtain credentials on attributes from an *issuer* and later prove possession or selectively disclose attributes to a *verifier*, preserving unlinkability. We use a Pointcheval-Sanders (PS) signature-based anonymous credential scheme [30], summarized below. We discuss why OrionLink chooses PS signatures in Section VI.

1) Pointcheval-Sanders Signature:

- $pp \leftarrow \mathsf{PS.Setup}(1^\lambda)$: Given a security parameter λ, select a type-3 bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ with generators $g_1 \in \mathbb{G}_1$, $g_2 \in \mathbb{G}_2$, where no efficient homomorphism exists between $\mathbb{G}_1$ and $\mathbb{G}_2$. The public parameters are $pp = (e, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2)$.
- $(sk, vk) \leftarrow \mathsf{PS.KeyGen}(pp, n)$: For n attributes, randomly select $x, y_1, ..., y_n \xleftarrow{\$} \mathbb{Z}_p$, compute $X = g_1^x, Y_i = g_1^{y_i}, \bar{X} = g_2^x, \bar{Y}_i = g_2^{y_i}$ for $i = 1, ..., n$. The keypair is secret key $sk = (x, y_1, ..., y_n, X)$, and verification key $vk = (Y_1, ..., Y_n, \bar{X}, \bar{Y}_1, ..., \bar{Y}_n)$.
- $\sigma \leftarrow \mathsf{PS.Sign}(sk, A)$: Given attribute vector $A = (A_1, \ldots, A_n)$, select a random $h \xleftarrow{\$} \mathbb{G}_1^*$ and compute signature $\sigma = (\sigma_1, \sigma_2) = (h, h^{x + \sum_{i=1}^n A_i y_i})$.
- $(0/1) \leftarrow \mathsf{PS.Verify}(vk, \sigma, A)$: Parse σ as (σ_1, σ_2), check $e(\sigma_2, g_2) = e(\sigma_1, \bar{X}) \prod_{i=1}^n e(\sigma_1, \bar{Y}_i)^{A_i}$.

A key property of PS signatures is *unlinkability:* a PS signature $\sigma = (\sigma_1, \sigma_2)$ can be randomized by sampling $k \xleftarrow{\$} \mathbb{Z}_p$ and computing $\sigma' = (\sigma_1^k, \sigma_2^k)$, which remains a valid signature on the same attributes.

2) Anonymous Credential Protocol: The protocol adopts the same setup and key generation procedures as the PS signature.

- $pp \leftarrow \mathsf{AC.Setup}(1^\lambda)$: Run $\mathsf{PS.Setup}(1^\lambda)$ to generate the public parameters pp.
- $(sk, vk) \leftarrow \mathsf{AC.KeyGen}(pp, n)$: Run $\mathsf{PS.KeyGen}(pp, n)$ to generate the issuer's secret key sk and verification key vk.

The protocol proceeds in three phases: Issuance, Presentation, and Verification.

Issuance. $(A, \sigma) \leftarrow \mathsf{AC.IssueCred}(pp, sk, A)$. The prover obtains a credential from the issuer for attribute vector $A = (A_h, A_p)$, hiding private attributes $A_h = (A_1, ..., A_j)$ while disclosing public attributes $A_p = (A_{j+1}, ..., A_n)$:

- $(d, \pi_1) \leftarrow \mathsf{PrepareBlindSign}(vk, A_h)$: Prover selects $d \xleftarrow{\$} \mathbb{Z}_p^*$, generates a non-interactive zero-knowledge (NIZK) proof $\pi_1 = \mathsf{PoK}\{(d, A_h) : C = g_1^d \prod_{i=1}^j Y_i^{A_i} \wedge \phi_1(A_h) = 1\}$ that hidden attributes (A_h) satisfy relation ϕ_1.
- $\sigma' \leftarrow \mathsf{BlindSign}(sk, A_p, \pi_1)$: Issuer verifies π_1, extracts C, selects $r \xleftarrow{\$} \mathbb{Z}_p$, and computes the signature $\sigma' = (g_1^r, (XC \prod_{i=j+1}^n Y_i^{A_i})^r)$, then sends it to the prover.
- $\sigma \leftarrow \mathsf{UnblindSign}(d, \sigma')$: Prover computes $\sigma = (\sigma_1', \sigma_2' {\sigma_1'}^{-d})$ to obtain the final credential σ.

Presentation. $(A_p', \sigma', \pi_2, \phi) \leftarrow \mathsf{AC.PresentCred}(vk, \sigma, A)$. To prove possession of a credential, the prover re-randomizes σ and selectively discloses public attributes (A_p'), generating NIZK proof π_2 that hidden attributes (A_h') satisfy relation ϕ. Specifically, it samples $k, t \xleftarrow{\$} \mathbb{Z}_p^*$ and computes

$$\sigma' = (\sigma_1', \sigma_2') = (\sigma_1^k, (\sigma_2 \sigma_1^t)^k).$$

The prover then generates a NIZK proof

$$\pi_2 = \mathsf{PoK}\{(t, A_h') : \mathsf{PS.Verify}(vk, (\sigma_1', \sigma_2' {\sigma_1'}^{-t}), A) \wedge \phi_2(A_h') = 1\},$$

which demonstrates knowledge of a valid signature on the entire attribute vector A, while keeping hidden attributes A'_h secret.

Verification. $(0/1) \leftarrow \mathsf{AC.VerifyCred}(vk, A'_p, \sigma', \pi_2, \phi)$. The verifier checks the NIZK proof π_2 to ensure that the credential is valid under the public attributes A'_p and that the hidden attributes A'_h satisfy the relation ϕ_2.

When no attribute is hidden ($A'_h = \emptyset$), π_2 degenerates to revealing t itself, and the verifier directly checks $\mathsf{PS.Verify}(vk, (\sigma'_1, \sigma'_2 \cdot \sigma'^{-t}_1), A) = 1$. This protocol satisfies *correctness*, *unlinkability*, *blindness*, and *unforgeability*, for which we provide formal definitions in Appendix G-B.

III. Threat Model and Target Properties

This section first introduces the threat model and then outlines the security and privacy properties, as well as the security assumptions, targeted by our proposed framework.

A. Threat Model

We consider a setting with malicious users and malicious RPs, as well as honest-but-curious identity brokers and IdPs. Corruption is static, and the IdP colludes neither with the broker nor with any RP. The broker may share its view with corrupted RPs while still following the protocol.

- **Malicious Users.** An adversary may control a user and attempt to impersonate another honest user to authenticate to honest RPs.
- **Malicious RPs.** An adversary may control one or more RPs and attempt to impersonate honest RPs to obtain user authorization or retrieve user data from the broker without proper consent. Colluding malicious RPs may further attempt to link a user's identity or activities across multiple services.
- **Honest-but-Curious IdP.** The IdP is assumed to follow the protocol correctly but may attempt to infer additional information, such as tracking a user's behavior across different RPs.
- **Honest-but-Curious Broker.** The broker is assumed to behave honestly due to reputational and operational incentives but may passively collect and analyze all authentication and authorization data it observes, including information provided by IdPs. For example, the broker may attempt to reconstruct users' behavioral patterns across multiple RPs and IdPs. In addition, mismanagement or misconfiguration may inadvertently expose user data to unauthorized RPs or third parties (see Appendix B-B for details).

B. Target Properties

OrionLink aims to preserve the standard security and privacy guarantees of brokered SSO while achieving the stronger goal of *broker obliviousness*. The ideal functionality $\mathcal{F}_{\mathsf{OrionLink}}$, shown in Figure 4, captures the desired security and privacy guarantees by structuring SSO interactions into a sequence of well-defined stages and explicitly specifying the information revealed at each stage.

Standard SSO Security. *OrionLink* satisfies the following standard SSO security properties:

- **Authentication Correctness.** An adversary cannot impersonate an honest user or an honest RP to successfully authenticate through an honest IdP.
- **Authorization Correctness.** An adversary cannot obtain or use protected resources without a valid authorization explicitly granted by the user to the RP.
- **Session Integrity.** An honest RP accepts a login result or associated authorization credentials only if the user has intentionally authenticated that RP within the same protocol execution.

Broker Obliviousness. The broker learns no information beyond the explicitly permitted metadata and the length of transferred user information, as specified by the BrokerView outputs in Figure 4. We formalize this property in Definition H.1.

Unlinkability and Untraceability. The RP remains anonymous to the IdP, preventing the IdP from learning which RP a user interacts with, as formalized in Definition H.2. In addition, user identifiers are unlinkable across RP–broker pairs, protecting users from cross-domain tracking, as formalized in Definition H.3.

Compatibility & Usability. OrionLink remains deployable within existing brokered SSO ecosystems without requiring client-side modifications, while preserving the broker's assistive functionalities, as discussed in Section VI.

C. Assumptions

Our design is based on the following security and operational assumptions:

- **Broker Visibility.** It is acceptable for an IdP to learn an RP's broker, as brokers typically serve multiple RPs (see Appendix B-A). This does not compromise RP anonymity. Accordingly, we do not require additional network-layer anonymity mechanisms (e.g., Tor [31]) to mitigate IP-based linkability, improving usability and deployability.
- **Unique User Identifier.** We assume that the unique identifier (uid) is the sole uniquely identifying user attribute maintained by the IdP.

IV. OrionLink

OrionLink is a privacy-enhancing brokered SSO system, involving four entities: the user, the RP, the IdP, and the broker. We describe OrionLink based on the standard Brokered SSO workflow of Section II-A, which is compatible with common SSO protocols, including OIDC [1] and SAML 2.0 [32].

A. Overview

The simplest route to broker obliviousness is end-to-end encryption between the RP and the IdP, with the broker reduced to a forwarder. This *strawman* hides user data from the broker, but it does not by itself hide the identities used to establish the channel. In a conventional end-to-end encrypted design, the RP and the IdP still authenticate using long-term or otherwise linkable credentials. As a result, the user-RP authorization relationship remains linkable and traceable,

$\mathcal{F}_{\mathsf{OrionLink}}$

Notation. Let D denote the RP identity (e.g., domain), cid an RP identifier, $acid$ a session-specific RP pseudonym, uid a global user identifier, uid_{rp} an RP-specific user pseudonym, $info$ authentication credentials and user attributes, and sid a session identifier.

- **On receiving** $(\mathsf{RegisterRP}, RP, D)$:
  1) Sample a fresh random identifier cid. Set $\mathsf{RPReg}[cid] := D$ and record the registrant $\mathsf{Owner}[cid] := RP$.
  2) Output $(\mathsf{Registered}, cid)$ to RP, $(\mathsf{Registered}, cid, D)$ to IdP, and $(\mathsf{BrokerView}, cid, D)$ to $Broker$.
- **On receiving** $(\mathsf{InitLogin}, RP, sid, cid, acid)$:
  1) If $\mathsf{RPReg}[cid] = \bot$ or $\mathsf{Sess}[sid] \neq \bot$, abort.
  2) If RP is honest, replace the supplied $acid$ with a fresh random one.
  3) If RP is corrupted, abort unless $\mathsf{Owner}[cid]$ is corrupted. Then send $(\mathsf{Judge}, sid, cid, acid)$ to the adversary and abort unless it replies accept, in which case adopt the supplied $acid$.
  4) Set $\mathsf{Sess}[sid] := (\mathsf{pending}, cid, acid)$.
  5) Output $(\mathsf{LoginCtx}, sid, acid)$ to RP, $(\mathsf{LoginReq}, sid, acid)$ to IdP, and $(\mathsf{BrokerView}, sid, cid, acid)$ to $Broker$.
- **On receiving** $(\mathsf{ShowRP}, RP, sid, cid, D)$:
  1) If $\mathsf{Sess}[sid] \neq (\mathsf{pending}, cid, \cdot)$ or $\mathsf{RPReg}[cid] \neq D$, abort.
  2) Output $(\mathsf{ShowRP}, sid, D)$ to $User$.
- **On receiving** $(\mathsf{Authorize}, User, sid, uid, decision)$:
  1) If $\mathsf{Sess}[sid] \neq (\mathsf{pending}, cid, acid)$ for some $(cid, acid)$, abort.
  2) If $decision = \mathsf{approve}$, update $\mathsf{Sess}[sid] := (\mathsf{approved}, cid, acid)$, otherwise abort.
  3) Output $(\mathsf{Consent}, sid, uid)$ to IdP.
- **On receiving** $(\mathsf{FinalizeLogin}, IdP, sid, uid, acid, info)$:
  1) If $\mathsf{Sess}[sid] \neq (\mathsf{approved}, cid, acid)$ with the same $acid$ as supplied, abort. Otherwise parse the recorded cid.
  2) If $\mathsf{UID}[uid, cid] = \bot$, sample fresh uid_{rp} and set $\mathsf{UID}[uid, cid] := uid_{rp}$.
  3) Output $(\mathsf{AuthOK}, sid, uid_{rp}, info)$ to RP, and $(\mathsf{BrokerView}, sid, uid_{rp}, |info|)$ to $Broker$.
  4) Set $\mathsf{Sess}[sid] := \mathsf{done}$.

Fig. 4: Ideal functionality $\mathcal{F}_{\mathsf{OrionLink}}$.

which breaks our security targets in §III-B. A second design axis, *anonymous endpoints*, is therefore unavoidable.

OrionLink couples anonymous authentication with the encrypted channel through two mechanisms. (*a*) *Anonymous pseudonym derivation* replaces long-term RP identifiers in IdP-facing interactions with fresh session-specific pseudonyms, while mapping global user identifiers to stable, RP-specific pseudonyms. (*b*) *Anonymous authentication key agreement (AAKA)* lets an anonymized RP and the IdP derive a session key K_S over the broker without exposing any long-term key or stable party identity. We detail the construction of both mechanisms in §IV-B.

Flow Overview (Fig. 5). At a high level, OrionLink proceeds in four phases as outlined below. We defer the full protocol specification to §IV-D.

- *Registration* (①–④). The RP registers its identity D at the IdP, which verifies the RP and issues an RP identifier cid together with an anonymous credential $\mathcal{DC}$. The RP then registers $(cid, \mathcal{DC})$ at the broker. This phase mirrors the standard brokered SSO registration flow, with the only difference being that the RP additionally obtains the anonymous credential $\mathcal{DC}$ at the IdP for use in subsequent logins.
- *Login Request* (⑤–⑥). On a user login, the RP derives a session pseudonym $acid$ from $\mathcal{DC}$, generates a fresh session key K_S, and sends $(cid, acid)$ to the broker, which formats and forwards $(acid)$ to the IdP. The RP still reaches multiple IdPs through the broker's unified authentication interface, preserving one of the broker's key assistive roles.
- *Auth & Consent* (⑦). The user independently verifies that $acid$ commits to D, which is displayed on the consent page, and then authenticates and grants consent at the IdP.
- *Login Response* (⑧–⑪). The IdP completes the AAKA to fix K_S, derives the RP-specific user pseudonym $auid$, and encrypts the user info as $\mathsf{info}_{Enc} = \mathsf{Enc}_{K_S}(\mathsf{info})$. It returns $(auid, \mathsf{info}_{Enc})$ through the broker over an E2EE channel. The RP unblinds $auid$ into uid_{rp} and decrypts info under K_S. The RP then returns uid_{rp} to the broker, which records it as the stable per-RP user pseudonym for user management.

Notations: We use $\mathsf{Enc}_K(\cdot)$ and $\mathsf{Dec}_K(\cdot)$ to denote symmetric authenticated encryption and decryption under key K, respectively, and $\mathsf{AEAD.Enc}_K(\cdot,\cdot)$ and $\mathsf{AEAD.Dec}_K(\cdot,\cdot)$ to denote authenticated encryption with associated data (AEAD). We denote $\mathsf{KDF}(\cdot)$ as a key derivation function, and $\mathsf{Sig}sk(\cdot)$ as a digital signature generated using the signing key sk.

B. Design

This section presents the construction of *anonymous pseudonym derivation* and *AAKA*, which together realize the

security and privacy targets of §III-B.

To facilitate cryptographic operations on the client-side without requiring browser extensions, we define a *Trusted Computation Agent (TCA)*. The TCA is a JavaScript component loaded into the user's browser via an `iframe` from a publicly accessible, auditable source. It assists the user in performing essential computational and verification steps, such as verifying credentials and deriving keys. The design draws inspiration from [20].

Remark. Revealing the RP's identity to the broker is acceptable and does not violate our security properties (see Appendix F). Hiding RP identity from the broker is left for future work under stronger assumptions.

1) Anonymous Pseudonym Derivation: To achieve identity obliviousness, OrionLink must ensure two properties: (1) *RP pseudonym derivation:* the IdP should authenticate the RP using a session-specific RP pseudonym without learning user–RP relationships, preventing cross-domain tracking. (2) *User pseudonym derivation:* each user should be identified by a unique, RP-specific pseudonym, which prevents identity linkability by the RP and the broker, and preserves the session consistency needed for broker-side data management. While prior work [20]–[22], [33], [34] has addressed this in standard SSO, we adapt these concepts for the brokered model.

RP pseudonym derivation. An RP's identity consists of two components. The first is a human-readable identity descriptor D, such as a domain name, which can be directly inspected by the user to verify that the RP is indeed the intended party to be authorized. The second is a long-term identity secret rsk_{IK}, held exclusively by the RP, which is used to prove possession of a registered identity to the IdP, such as the `client_secret` in OIDC flow.

During SSO, the RP's identity must be verified twice:

- *User-facing verification*. The user verifies the RP's identity on the frontend and decides whether to grant authorization which relies on presenting the identity descriptor D to the user (step ⑦).
- *Token Issuance*. The RP anonymously proves possession of the corresponding rsk_{IK} to the IdP, thereby establishing identity validity without revealing its real identity. To ensure *session integrity*, the IdP must further verify that the entity requesting the authentication credentials is the same entity that the user has authorized (step ⑧).

Since users do not need to verify rsk_{IK}, we issue two separate credentials so that the user only verifies D on the frontend. Building on the anonymous credential scheme from Section II-D, the RP obtains two credentials from the IdP during registration:

- *Domain Credential ($\mathcal{DC}$)*: A PS signature on the public attribute D, binding the RP to its registered identity.
- *Secret Credential ($\mathcal{SC}$)*: A PS signature over both the public attribute D and the private attribute rsk_{IK}, which is also used in the AAKA protocol for key agreement.

For *User-facing verification*, the RP derives a pseudonymous identifier $k, t \xleftarrow{\$} \mathbb{Z}_p^*, acid := (\sigma_1^k, (\sigma_2 \sigma_1^t)^k)$ from $\mathsf{AC.PresentCred}(\mathcal{DC})$ (step ⑤) and the broker redirects it to the IdP window. Since D is fully disclosed to the user at consent time, this presentation is a fully disclosed one in the sense of Section II-D, so the accompanying proof π_D is realized by the randomization factor t itself, which the RP sends alongside $acid$ and the broker forwards to the TCA. We use TCA on behalf of the user to derandomize $acid$ with t, verify D against it, and display D on the consent page (step ⑦). Upon user confirmation, the consent result is sent to the IdP to obtain the authentication credential. For *Token Issuance*, the RP generates a NIZK proof π_2 via $\mathsf{AC.PresentCred}(\mathcal{DC}, \mathcal{SC})$ (step ⑤) to prove its link to $acid$ and its possession of rsk_{IK}. The IdP verifies it with $\mathsf{AC.VerifyCred}$ (step ⑧), ensuring RP anonymity and consistency without revealing D or rsk_{IK}.

User pseudonym derivation. To derive unlinkable user pseudonyms, we adopt the OPRF-based scheme in Section II-B, ensuring that each pseudonym is unique per RP-Broker while remaining unlinkable across domains. Each user (uid) and each RP (cid) is associated with a unique identifier assigned by the IdP, from which RP-specific user pseudonyms are derived in the form $uid_{rp} = cid^{uid}$.

During registration, the IdP samples $cid := h, h \xleftarrow{\$} \mathbb{G}_1^*$, and embeds h into the signature $\sigma = (cid, \sigma_2)$ over the RP's identity descriptor D .

During login phase, the RP generates a session-specific pseudonym $acid = (\sigma_1', \sigma_2') = (cid^k, (\sigma_2 \cdot cid^t)^k)$, which is similar to the RP blinds its identifier by computing $(k, cid^k) \leftarrow \mathsf{Blind}(cid)$ for fresh randomness k. The IdP performs a blind evaluation and computes $auid = \mathsf{BlindEval}(uid, \sigma_1')$ (step ⑧). Upon receiving $auid$, the RP removes the blinding factor and computes $\mathsf{Unblind}(k, auid) = auid^{1/k} = h^{uid}$, obtaining the RP-specific user identifier uid_{rp} (step ⑩).

This construction allows the RP to later derive the RP-specific user identifier uid_{rp} without learning the user's global identifier uid, while ensuring that the IdP does not learn the RP identifier cid during authentication.

2) Anonymous Authentication Key Agreement: After the IdP authenticates the RP and obtains user consent, it must forward the user's authorization credentials and personal information to the RP via the broker, which may expose sensitive user data to the broker. To prevent this, OrionLink establishes an E2EE channel between the RP and the IdP using a variant of the X3DH key agreement protocol [35]. The RP and the IdP derive a fresh session key for each session from their long-term identity keys, the RP's ephemeral key, and the IdP's signed prekey, which is refreshed once per epoch. Because the IdP reuses its signed prekey within an epoch, AAKA provides forward secrecy only across prekey epochs rather than at session granularity. Appendix F details this limitation and describes an upgrade to session-level forward secrecy. The broker only forwards ciphertexts and has no access to plaintext user data.

To resist man-in-the-middle (MitM) attacks while hiding the RP's real identity, the RP needs to maintain a long-term identity key $(\mathsf{rsk}_{IK}, \mathsf{rpk}_{IK})$ and derives a fresh, anonymous identity key $(\mathsf{rsk}'_{IK}, \mathsf{rpk}'_{IK})$ for each session. During registra-

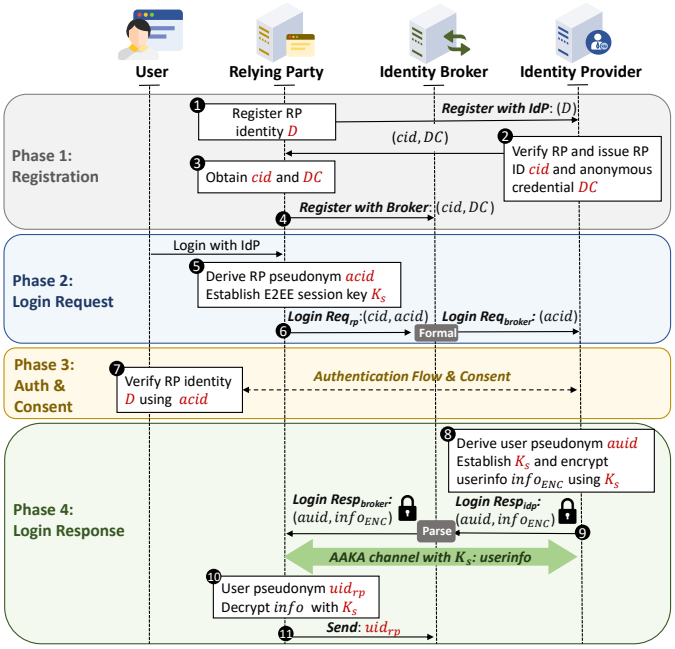


Fig. 5: Overview of OrionLink.

tion, the RP embeds its long-term secret identity key rsk_{IK} into $\mathcal{SC}$ as a private attribute, with corresponding public key $\mathsf{rpk}_{IK} = g_1^{\mathsf{rsk}_{IK}}$.

For each session, the RP randomizes its public key as $r \xleftarrow{\$} \mathbb{Z}_p, \mathsf{rpk}'_{IK} = g_1^{r+\mathsf{rsk}_{IK}}$ and proves its validity via NIZK (step ⑤). By anonymously verifying the RP, the IdP checks that rpk'_{IK} is correctly derived from the RP's long-term identity secret rsk_{IK}, establishing possession of a registered identity. The verified key rpk'_{IK} is then used as the RP's identity key in the key agreement protocol and combined with the RP's ephemeral key and the IdP's signed prekey to derive the session key $K_S \leftarrow \mathsf{X3DH}_{\mathsf{RP}}(\mathsf{rsk}'_{IK}, \mathsf{rsk}_E, \mathsf{ipk}_{IK}, \mathsf{ipk}_E)$ while the RP computes K_S using the IdP's identity key ipk_{IK} obtained during the initial registration (step ⑤). The IdP runs the mirrored responder routine $\mathsf{X3DH}_{\mathsf{IdP}}$ of Figure 9 on the corresponding keys, so both sides derive the same K_S. The IdP encrypts authorization credentials and personal information using the session key K_S, while keeping attribute names in plaintext. This allows the broker to map identities from diverse IdPs into a unified user representation.

C. Privacy without Sacrificing Broker Utility

With both identifiers and user data hidden from the broker, one immediate question follows:

Does broker obliviousness come at the cost of broker utility?

The answer is largely no. OrionLink restricts only the broker functions that require plaintext values and preserves those that depend on protocol structure and metadata. The message pattern between the RP and the IdP is unchanged, so the broker still provides a unified authentication interface, protocol translation, and a single upgrade point for both sides (*Unified Authentication Interface* ✓, *Scalability* ✓). Attribute names

$\Pi_{\mathsf{OrionLink}}$

Phase 0: Initialization

Set the public parameters $pp \leftarrow (e, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2)$.

- **IdP:** (a) a long-term identity key pair $(\mathsf{isk}_{IK}, \mathsf{ipk}_{IK})$; (b) a credential key pair $(\mathsf{isk}_c, \mathsf{ivk}_c)$.
- **RP:** A long-term identity key pair $(\mathsf{rsk}_{IK}, \mathsf{rpk}_{IK})$.

Phase 1: Registration

- **RP: Start Registration** to the IdP.
  1) Execute $(d, \pi_1) \leftarrow \mathsf{PrepareBlindSign}(\mathsf{ivk}_c, D)$
  2) Send (D, π_1) to the IdP.

  On receiving $(cid, \sigma, \hat{\sigma})$ from the IdP:
  1) $\mathsf{UnblindSign}(d, \bar{\sigma}_1) \rightarrow \bar{\sigma}$, and set $\mathcal{DC} = (D, \sigma)$ and $\mathcal{SC} = (D, \mathsf{rsk}_{IK}, \bar{\sigma})$.
  2) Send $(cid, \mathcal{DC})$ to the broker.
- **IdP: On receiving** (D, π_1) from the RP.
  1) Execute $\sigma \leftarrow \mathsf{PS.Sign}(\mathsf{isk}_c, D)$ and $\hat{\sigma} \leftarrow \mathsf{BlindSign}(\mathsf{isk}_c, D, \pi_1)$.
  2) Extract cid from σ.
  3) Send $(cid, \sigma, \hat{\sigma})$ to the RP.
- **Broker: On receiving** $(cid, \mathcal{DC})$ from the RP.
  1) Register and store $(cid, \mathcal{DC})$.

Phase 2: Login Request

- **RP: On receiving** a login request from the user.
  1) Derive a session-specific pseudonym $acid$, and $\pi_D := t_1$.

  $$(D, acid, \pi_D) \leftarrow \mathsf{AC.PresentCred}(\mathsf{ivk}_c, \sigma, D)$$

  2) Execute AnonyAKA to establish a session key K_S and generate

  $$M = (\bar{\sigma}', \pi_2, \mathsf{rpk}'_{IK}, \mathsf{rpk}_E, \mathsf{msg}_{init})$$

  3) Send $(cid, acid, \pi_D, M)$ to the broker.
- **Broker: On receiving** $(cid, acid, \pi_D, M)$ from RP.
  1) Records $(cid, acid)$ and retrieves D from $\mathcal{DC}$.
  2) Construct an authentication request with $(acid, M)$ and forward it to the IdP.
  3) Forward (D, π_D) to the TCA in the user's browser.

Phase 3: User Authentication & Consent

- **User (via TCA): On receiving** $acid$ from IdP and (D, π_D) from the broker via cross-window messaging.
  1) Parse $acid = (\sigma'_1, \sigma'_2)$ and $\pi_D = t_1$, then execute $\mathsf{AC.VerifyCred}(\mathsf{ivk}_c, D, acid, \pi_D)$ to verify RP's identity D.
  2) If accepted, send consent to IdP.

Phase 4: Login Response

- **IdP: On receiving** consent from the user.
  1) Execute AnonyAKA to verify π_2 and derive the session key K_S.
  2) Encrypt $\mathsf{info}_{Enc} := \mathsf{Enc}_{K_S}(\mathsf{info})$.
  3) Derive an RP-specific pseudonym $auid = \mathsf{BlindEval}(uid, \sigma'_1)$.
  4) Send $(auid, \mathsf{info}_{Enc})$ to the broker.
- **Broker: On receiving** $(auid, \mathsf{info}_{Enc})$ from the IdP.
  1) Forward $(auid, \mathsf{info}_{Enc})$ to the RP

  On receiving uid_{rp} from the RP.
  1) Record (cid, uid_{rp}) as the per-RP user pseudonym for this session.
- **RP: On receiving** $(auid, \mathsf{info}_{Enc})$ from the broker.
  1) Unblind $uid_{rp} \leftarrow \mathsf{Unblind}(k_1, auid)$
  2) Decrypt $\mathsf{info} := \mathsf{Dec}_{K_S}(\mathsf{info}_{Enc})$.
  3) Send uid_{rp} to the broker.

Fig. 6: The *OrionLink* Protocol $\Pi_{\mathsf{OrionLink}}$.

stay in plaintext and the per-RP pseudonym $uid_{rp} = h^{uid}$ remains stable, so the broker can still map heterogeneous IdP claims into a unified representation and maintain user-RP relationships (*Identity Normalization* ≈✓). Plaintext refresh metadata (Section IV-E2) still supports token validation, re-

fresh, and audit (*Centralized Security Enforcement* $\approx$✓).

Limitations. Broker obliviousness carries a functionality cost. Without plaintext values or token payloads, the broker cannot offer value-dependent services such as attribute-value normalization, broker-side attribute-based access control (ABAC), user profiling, or content-based threat analytics. We accept this cost because OrionLink targets cross-organizational public brokers, which carry 71% of observed brokered SSO flows (Appendix B-A), where user privacy takes priority. Enterprise deployments that value centralized logging or ABAC can selectively disclose attribute values to the broker, explicitly relaxing broker obliviousness for those attributes only. OrionLink also adds modest key-management overhead for RPs.

D. System Details

The OrionLink protocol consists of four phases: *Registration*, *Login Request*, *User Authentication & Consent*, and *Login Response*. Figure 5 shows the overview of OrionLink. Figure 6 specifies the full OrionLink protocol. We omit certain engineering details that do not affect the system.

1) Initialization: In this stage, the IdP, the RP, and the broker generate the required key materials and public parameters. **Public parameters:** Let λ be the security parameter. Select a type-3 bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ with generators $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$. The public parameters are denoted as $pp \leftarrow (e, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2)$.

- The IdP generates the following key materials :
  (*a*) a long-term identity key pair $(\mathsf{isk}_{IK}, \mathsf{ipk}_{IK})$ for the AAKA protocol with the RP, where $\mathsf{isk}_{IK} \xleftarrow{\$} \mathbb{Z}_p$ and $\mathsf{ipk}_{IK} = g_1^{\mathsf{isk}_{IK}}$;
  (*b*) a credential key pair $(\mathsf{isk}_c, \mathsf{ivk}_c)$: the IdP chooses $x_1, y_1 \xleftarrow{\$} \mathbb{Z}_p$ for $\mathcal{DC}$ and $x_2, y_{21}, y_{22} \xleftarrow{\$} \mathbb{Z}_p$ for $\mathcal{SC}$, then computes $X_1 = g_1^{x_1}, Y_1 = g_1^{y_1}, \bar{X}_1 = g_2^{x_1}, \bar{Y}_1 = g_2^{y_1}$ and $X_2 = g_1^{x_2}, Y_{21} = g_1^{y_{21}}, Y_{22} = g_1^{y_{22}}, \bar{X}_2 = g_2^{x_2}, \bar{Y}_{21} = g_2^{y_{21}}, \bar{Y}_{22} = g_2^{y_{22}}$. The IdP sets $\mathsf{isk}_c \leftarrow (x_1, y_1, X_1, x_2, y_{21}, y_{22}, X_2)$ and $\mathsf{ivk}_c \leftarrow (Y_1, \bar{X}_1, \bar{Y}_1, Y_{21}, Y_{22}, \bar{X}_2, \bar{Y}_{21}, \bar{Y}_{22})$.
- RP key materials: A long-term identity key pair $(\mathsf{rsk}_{IK}, \mathsf{rpk}_{IK})$, where $\mathsf{rsk}_{IK} \xleftarrow{\$} \mathbb{Z}_p$ and $\mathsf{rpk}_{IK} = g_1^{\mathsf{rsk}_{IK}}$, for the AAKA protocol.

2) Registration: In this stage, the user and the RP register with the IdP. The IdP verifies the user's identity and assigns a unique identifier $uid \xleftarrow{\$} \mathbb{Z}_p$.

The RP registers with the IdP by submitting its identity key rsk_{IK} as a private attribute and its identity attribute D (e.g., domain) as a public attribute. The procedure is as follows:

- The RP selects a blinding factor $d \xleftarrow{\$} \mathbb{Z}_p$ and computes a Pedersen commitment $C := g_1^d \cdot Y_{21}^{\mathsf{rsk}_{IK}}$. It then generates a NIZK proof
$$\pi_1 = \mathsf{PoK}\{(d, \mathsf{rsk}_{IK}) : C = g_1^d \cdot Y_{21}^{\mathsf{rsk}_{IK}}\},$$
and sends (D, π_1) to the IdP.
- The IdP verifies the proof π_1. If valid, it sets $cid = h$ as the RP's unique identifier, where $h \xleftarrow{\$} \mathbb{G}_1^*$, and selects $r \xleftarrow{\$} \mathbb{Z}_p$. It then computes a signature on D as $\sigma = (\sigma_1, \sigma_2) = (h, h^{x_1 + y_1 \cdot D})$, and a blinded signature on D and rsk_{IK} as $\hat{\tilde{\sigma}} = (g_1^r, (X_2 \cdot C \cdot Y_{22}^D)^r)$. It returns $(cid, \sigma, \hat{\tilde{\sigma}})$ to the RP.
- The RP unblinds $\hat{\tilde{\sigma}}$ to obtain $\bar{\sigma} = (\hat{\tilde{\sigma}}_1, \hat{\tilde{\sigma}}_2 \hat{\tilde{\sigma}}_1^{-d})$, and derives the domain credential $\mathcal{DC} = (D, \sigma)$ and the secret credential $\mathcal{SC} = (D, \mathsf{rsk}_{IK}, \bar{\sigma})$.

Finally, the RP forwards $(cid, \mathcal{DC})$ to the broker. We omit the broker's response for brevity.

3) Login Request: In this phase, the RP receives the login request from the user, generates an RP pseudonym and the AAKA materials, and forwards them to the broker. The broker then constructs a standard SSO authorization request containing these materials and sends it to the IdP for authorization.

Pseudonym Derivation. The RP randomizes $\mathcal{DC}$ to derive the RP's session-specific pseudonym. It samples $k_1, t_1 \xleftarrow{\$} \mathbb{Z}_p^*$ and computes $acid := \sigma' = (\sigma_1', \sigma_2') = (\sigma_1^{k_1}, (\sigma_2 \sigma_1^{t_1})^{k_1})$. The RP also outputs $\pi_D := t_1$, the fully disclosed presentation proof that lets the user derandomize $acid$ and check it against D. The blinding factor k_1 is never released, so $acid$ remains unlinkable across sessions.

AAKA execution. The RP then runs the AnonyAKA protocol in Figure 9 to generate $M = (\pi_2, \bar{\sigma}', \mathsf{rpk}'_{IK}, \mathsf{rpk}_E, \mathsf{msg}_{init})$,which includes the anonymous authentication proof $(\bar{\sigma}', \pi_2, \mathsf{rpk}'_{IK})$ and key agreement payload $(\mathsf{msg}_{init}, \mathsf{rpk}_E)$.

The RP executes
$$(\pi_2, \bar{\sigma}', (\mathsf{rsk}'_{IK}, \mathsf{rpk}'_{IK})) \leftarrow \mathsf{GenProof}(\mathcal{SC}, acid, \mathsf{ivk}_c)$$
and obtains the proof: (i) a randomized credential signature $\bar{\sigma}'$ derived from $\mathcal{SC}$, (ii) an ephemeral identity key rpk'_{IK}, and (iii) a NIZK proof π_2. The proof guarantees that $acid$ and $\mathcal{SC}$ correspond to the same identity attribute D, and that rpk'_{IK} is correctly derived from rsk_{IK} embedded in $\mathcal{SC}$.

The RP then derives the session key K_S using its freshly generated ephemeral key rsk_E and the signed prekey ipk_E obtained from the IdP
$$K_S \leftarrow \mathsf{X3DH}_{\mathsf{RP}}(\mathsf{rsk}'_{IK}, \mathsf{rsk}_E, \mathsf{ipk}_{IK}, \mathsf{ipk}_E)$$
then obtains the initial message msg_{init} and an ephemeral key rpk_E, which enable the IdP to perform the key agreement. The RP forwards the user's login request, together with $(cid, acid, \pi_D, M)$ to the broker. Specifically, cid serves as an explicit identifier for the RP in this session, enabling the broker to store and manage future user data associated with the RP. The broker constructs an authentication request according to standard SSO requirements with $(acid, M)$ and sends it to the IdP. It retains π_D and delivers it to the TCA together with D in the consent phase. π_D must never reach the IdP, which could otherwise derandomize $acid$ with t_1 and match it against every registered RP descriptor to recover the RP's identity.

4) User Authentication & Consent: The user must verify the RP's identity and determine whether to grant authorization.

The user must first successfully log in to the IdP. Then the IdP loads TCA in the frontend window to help the user verify RP legitimacy. TCA retrieves (D, π_D) from the broker via cross-window messaging, parses π_D =

t_1, and checks $(0/1) \leftarrow \mathsf{AC.VerifyCred}(\mathsf{ivk}_c, D, acid, \pi_D)$, which derandomizes $acid = (\sigma'_1, \sigma'_2)$ with t_1 and verifies $\mathsf{PS.Verify}(\mathsf{ivk}_c, (\sigma'_1, \sigma'_2 \cdot \sigma'^{-t_1}_1), D) = 1$. If successful, D is shown on the consent page. If the user consents to authorize the RP, the IdP proceeds with the following steps.

5) Login Response: This phase establishes a session key between the RP and IdP, enables anonymous RP authentication, generates an RP-specific user pseudonym, issues authentication credentials and user information info.

Pseudonym Derivation. The IdP parses $acid = \sigma' = (\sigma'_1, \sigma'_2)$, where $\sigma'_1 = cid^k_1$. The IdP computes the user's RP-specific pseudonym for the RP as $auid = \mathsf{BlindEval}(uid, \sigma'_1)$

AAKA execution. The IdP executes AnonyAKA to verify proof π_2:

$$(0/1) \leftarrow \mathsf{VerifyProof}(\pi_2, \sigma', \bar{\sigma}', \mathsf{rpk}'_{IK}, (\mathsf{isk}_c, \mathsf{ivk}_c))$$

confirming the RP's validity, and derives the shared session key K_S. The IdP encrypts the user's sensitive information under the session key K_S as $\mathsf{info}_{Enc} := \mathsf{Enc}_{K_S}(\mathsf{info})$ and forwards the pair $(auid, \mathsf{info}_{Enc})$ to the RP via the broker. The RP unblinds the user pseudonym as $uid_{rp} = \mathsf{Unblind}(k_1, auid)$, decrypts info_{Enc} with K_S, completing the SSO flow. Finally, the RP delivers the unblinded uid_{rp} to the broker, which records it under cid as the stable per-RP user pseudonym (step (11)). Only the RP holds the blinding factor k_1, so this delivery is what gives the broker the uid_{rp} that BrokerView in Figure 4 permits it to observe.

E. Instantiation of OrionLink

We instantiate OrionLink based on the widely adopted OIDC Authorization Code flow [1].

Instantiating OrionLink in practice poses several challenges. In particular, OrionLink must be carefully integrated with the OIDC specifications [12], while preserving the broker's assistive functionalities described in Section II-A, ensuring that it remains practically efficient and deployable. This fills a critical but often neglected gap in prior privacy-preserving SSO systems [20]–[23]. To this end, we introduce two features into the OIDC-based instantiation: (*a*) *private authorization management:* allows users to review and revoke their RP accesses without disclosing authorization relationships to the IdP; (*b*) *forward-secure token refresh:* enables the broker to securely assist RPs in maintaining the validity of user tokens through refresh tokens.

1) Private Authorization Management: While IdP-to-RP anonymous authentication prevents the IdP from tracking users' authorization choices, users nonetheless require visibility and control over their authorized RPs (e.g., for access review or revocation).

We address this with the OPRF-based mechanism mentioned in Section II-B that allows the user to derive a high-entropy key K_m from a short PIN in interaction with the IdP. The derived key K_m is used on the frontend to encrypt the RP identity descriptor D, bind it to the RP pseudonym $acid$, and store the encrypted result $(acid, Enc_{K_m}(D))$ at the IdP. When the user later reviews or revokes an authorization, the same procedure is executed to re-derive the key and decrypt D for RP identity verification. The corresponding $acid$ is then sent to the IdP to trigger the requested authorization management operation. The details of the algorithm are as follows.

During the *User Authentication & Consent* phase (step (7) in Figure 5), if the user approves the authorization for the RP, a short PIN is entered. TCA then runs an OPRF with the IdP on behalf of the user to derive a management key K_m from the PIN:

1) TCA computes $r', \alpha = \mathsf{Blind}(\mathsf{PIN})$ and sends α to the IdP;
2) The IdP evaluates $\beta = \mathsf{BlindEval}(\mathsf{k}_{oprf}, \alpha)$ with its OPRF key k_{oprf} and returns β;
3) TCA computes $y = \mathsf{Unblind}(r', \beta)$, derives $K_m = \mathsf{KDF}(y)$, and encrypts the payload as $D_{\mathsf{Enc}} = \mathsf{Enc}_{K_m}(D)$.

This approach derives strong keys from short PINs, reducing user burden while maintaining confidentiality.

2) Forward-secure Token Refresh: In the OIDC workflow, the IdP issues three types of tokens: *ID tokens*, *access tokens*, and *refresh tokens* (see details in Appendix C). The broker manages *access token* lifecycles on behalf of the RP. If an *access token* expires, the broker uses the *refresh token* to request a new one from the IdP.

To preserve privacy against broker surveillance, we design a *key rotation mechanism* inspired by the double ratchet [36]. Each refreshed token is encrypted under a freshly derived session key, ensuring forward secrecy. Additionally, a Diffie–Hellman ratchet injects new randomness, achieving post-compromise security by preventing past or long-term key exposure from affecting future tokens. The full protocol details are described in Figure 11 in Appendix D. In practice, the RP pre-generates a sequence of ephemeral DH key pairs and sends the public keys to the broker. During operation, the RP simply requests access tokens from the broker. If an update is required, the broker uses the next DH key to negotiate a new session key with the IdP. The IdP then encrypts the refreshed access token under this key, and the broker returns it to the RP together with the material for updating K_S.

This mechanism allows the broker to continuously manage *access tokens*, without requiring RP involvement in every refresh operation, while ensuring that only the RP and IdP can access the token contents, preserving both functionality and end-to-end privacy.

F. Security Analysis

OrionLink is analyzed in the UC framework. The ideal functionality $\mathcal{F}_{\mathsf{OrionLink}}$ in Figure 4 captures both the security and the privacy goals, and Theorem IV.1 states that the real protocol $\Pi_{\mathsf{OrionLink}}$ securely realizes this functionality. Appendix H gives the simulation-based proof, formalizes each privacy goal as a standalone definition, and establishes each definition under the assumptions of Theorem IV.1 (Corollary H.1).

Theorem IV.1 (Security of OrionLink)**.** *Assume that (i) the Decisional Diffie–Hellman (DDH) problem is hard, (ii) the hash functions used in the protocol are modeled as random*

oracles, so that the X3DH*-derived session key is indistinguishable from random, (iii) the symmetric encryption scheme* Enc *is IND-CPA secure, (iv) the anonymous credential scheme satisfies correctness, unforgeability, blindness, and unlinkability, and (v) the DH-based OPRF satisfies correctness, consistency, and pseudorandomness. Then the protocol* $\Pi_{\mathsf{OrionLink}}$ *securely realizes the ideal functionality* $\mathcal{F}_{\mathsf{OrionLink}}$.

We now spell out what Theorem IV.1 implies for each target property of Section III-B. For every property we state how $\mathcal{F}_{\mathsf{OrionLink}}$ models it, which mechanism enforces it in the real protocol, and where its formal definition appears.

Standard SSO Security. We follow prior work [37] in defining standard SSO security and model these properties in $\mathcal{F}_{\mathsf{OrionLink}}$. An honest IdP issues a token only after a valid user consent and successful authentication of the RP, and the broker forwards the token exclusively to the corresponding RP session. We omit an explicit definition of user authentication correctness in $\mathcal{F}_{\mathsf{OrionLink}}$, since our protocol integrates the standard user authentication mechanisms of conventional SSO systems without modification, and inherit the property directly from standard SSO.

Broker Obliviousness. This property is captured by the leakage function BrokerView in Figure 4, which specifies the maximum information revealed to the broker during protocol execution. The broker observes only permitted metadata and the length of transferred user information. In the real protocol, this restriction is enforced by end-to-end encryption between the RP and the IdP (Definition H.1).

Untraceability. In $\mathcal{F}_{\mathsf{OrionLink}}$, the IdP observes only session-specific identifiers that are independent of the RP's long-term identity. The real protocol realizes this with freshly randomized anonymous credentials and zero-knowledge proofs in each login request, so user–RP authorization relationships remain untraceable at the IdP. In particular, the IdP issues and verifies the RP credential, so the joint issuer–verifier view is already in scope (Definition H.2).

Unlinkability. The RP and the broker observe only RP-specific user pseudonyms, derived via an OPRF as pairwise identifiers bound to the RP's anonymous identifier, so user identities remain unlinkable across RPs even when the broker and multiple RPs pool their views (Definition H.3).

V. Implementation and Evaluation

This section describes the implementation of the *OrionLink* prototype and presents a comprehensive evaluation of its performance overhead.

A. Implementation

We implement a prototype of OrionLink following the instantiation described in Section IV-E. The prototype comprises over 9,000 lines of code and is publicly available at https://github.com/Ooo0ption/Orionlink.git.

The system fully realizes all functionalities of the RP, IdP, and broker as specified in the OrionLink design. Its implementation consists of the following core components.

(1) *Complete OIDC Login Flow.* The prototype implements the full OIDC Authorization Code flow, including registration, login request, user authentication and consent, and login response. All protocol interactions and request/response constructions strictly follow the OIDC specification [12]. We adopt the BLS12-381 curve [38] for Pointcheval–Sanders (PS) signatures and issue and verify tokens as JSON Web Tokens (JWTs) [39]. All backend components are implemented in Go and leverage Cloudflare's CIRCL cryptographic library [40].

(2) *Trusted Computation Agent.* The TCA is a JavaScript component responsible for executing all user-side cryptographic operations, including anonymous credential verification and OPRF computations. It is implemented using the noble-curves [41] and facebook-ristretto255 [42] libraries.

(3) *Private Authorization Management.* Private authorization management is realized through user-side OPRF operations executed within the TCA, with corresponding server-side support deployed at the IdP using the Ristretto255 curve [43].

(4) *Token Refresh with Key Rotation.* The token refresh mechanism is inspired by the double ratchet protocol [36], providing forward secrecy across successive refresh operations.

B. Evaluation

All RP, broker, and IdP backend components are deployed as Docker containers on a single Ubuntu 24.04 server equipped with a 16-core 4.3 GHz CPU, 96 GB of RAM, and a 2 TB SSD.

Local Computational Overhead. Table II reports the local computational overhead of each privacy-enhancing stage in OrionLink, with results averaged over 1,000 runs. The *RP–broker* registration step is omitted, as it introduces no additional computation. The *Authorization Management* measurement captures the time for the TCA to decrypt domain information D on the IdP page during authorization management. The *Token Refresh* measurement reports the latency of a single refresh operation involving the RP, broker, and IdP.

Across all stages, the overhead remains minimal, indicating that OrionLink's privacy mechanisms incur negligible local computation cost. The total local computational overhead is approximately 67.50 ms.

End-to-End Performance. We evaluate the end-to-end latency of OrionLink under realistic user interactions and emulated wide-area network conditions. Latency is defined as the elapsed time from when the user clicks the "Login" button to the successful completion of authentication at the RP.

A login session consists of four steps: (1) redirecting to the IdP consent page after the user clicks "Login"; (2) user consent with frontend execution of the TCA; (3) authorization code exchange by the RP; (4) successful login completion.

We implement a standard brokered SSO baseline using the OIDC Authorization Code flow. Except for additional UI pages required by *OrionLink*, both systems follow similar webpage structures and interaction patterns. User interactions are automated using Playwright [44] to deterministically trigger click events. Table III reports the results under emulated round-trip time (RTT) network latencies of 0 ms, 40 ms, 100 ms, and 200 ms, with each result averaged over 100 runs.

TABLE II: Local computation overhead.

Stage	Operation	Time (ms)
Initialization	Key generation	46.59
Registration	RP registers to IdP	5.63
SSO	Pseudonym generation	2.58
	AnonyAKA execution	11.39
Extension	Authorization revoke	0.11
	Token refresh	1.20
Total		67.50

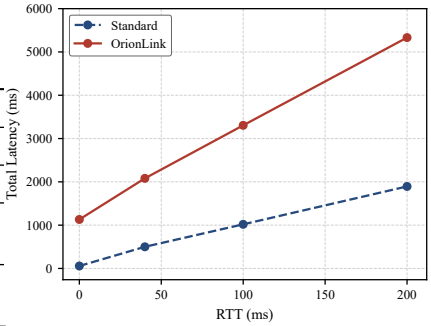


Fig. 7: User login latency (*OrionLink* vs. Standard).

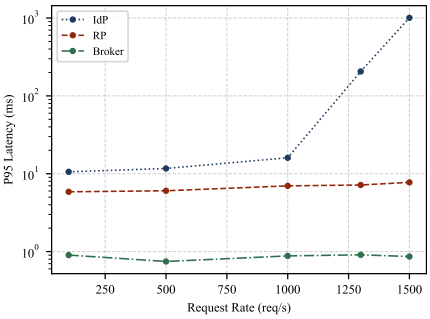


Fig. 8: Stress test of the AnonyAKA interface.

TABLE III: End-to-End performance of *OrionLink* and standard Brokered SSO (RTT=0 ms,40 ms,100 ms,200 ms).

Category	Stage	0 ms		40 ms		100 ms		200 ms	
		OrionLink	**Standard**	**OrionLink**	**Standard**	**OrionLink**	**Standard**	**OrionLink**	**Standard**
SSO Flow	Click "Login"	416.47	36.69	625.67	206.60	904.23	389.51	1365.34	697.95
	Consent check	53.01	–	207.28	–	422.99	–	776.94	–
	Authorization code exchange	122.27	19.80	595.88	293.23	1203.45	629.36	2215.37	1195.89
	Login complete (Total)	**1131.08**	**56.67**	**2080.63**	**500.01**	**3305.41**	**1019.12**	**5331.78**	**1894.04**
Extension	Authorization management	20.41	–	68.51	–	130.73	–	235.38	–
	Token refresh	2.62	–	123.57	–	272.53	–	521.15	–

Compared to the baseline, *OrionLink* introduces additional latency due to its privacy-enhancing mechanisms and the resulting increase in user interactions, page redirections, and frontend rendering overhead. These overheads primarily stem from additional login-phase redirections, frontend execution of the TCA during consent, and cross-window communication during authorization code exchange. As shown in Figure 7, the end-to-end latency of both systems increases approximately linearly with network delay. Under a typical network latency of 40 ms, a complete *OrionLink* authentication takes 2.08 s on average, with the privacy extensions contributing an additional 1.58 s. The overall login latency remains acceptable for interactive use. Future optimizations could further reduce latency by improving frontend efficiency (e.g., through WebAssembly [45]) and streamlining the interaction flow to reduce protocol rounds.

Stress Test. Local overhead analysis identifies AAKA execution as the most computationally intensive component of *OrionLink*. We therefore conduct stress tests on the AAKA interfaces jointly executed by the RP, broker, and IdP. As shown in Figure 8, under a load of 1,000 requests per second (RPS), the system maintains a P95 latency of approximately 10 ms, indicating stable and low-latency performance even under high load.

Mobile Evaluation. The TCA executes only a fixed set of cryptographic operations in the browser during one login. This set takes 32.4 ms on an iPhone 15 (iOS 18.7, Safari) versus 32.2 ms on a consumer laptop (AMD Ryzen 7 5800HS, 16 GB RAM, Edge), both medians over 100 runs, so mobile clients incur no measurable extra cost.

VI. Discussion

Choice of PS over BBS+. OrionLink builds on one structural property of PS signatures. The first component $\sigma_1 = h$ is a random group element chosen by the issuer, reused in OrionLink as the per-RP identifier $cid := h$. Re-randomizing the credential gives $\sigma_1' = h^{k_1}$, from which the IdP blindly evaluates $auid = (h^{k_1})^{uid} = (h^{uid})^{k_1}$ and the RP unblinds $uid_{rp} = h^{uid}$, so the RP credential and the OPRF-based pseudonym derivation of Section IV-B share one algebraic base. BBS+ does not expose an analogous independently chosen group-element component that can be directly reused as the pseudonym base, and would therefore require a separate pseudonym-derivation mechanism.

Compatibility with Other SSO Protocols. *OrionLink* is compatible with other federated SSO protocols, including SAML 2.0 [32] and WS-Federation [48]. At a high level, these protocols follow a common authentication workflow, as described in Section II-A, upon which OrionLink is built. Differences are largely confined to message formats and token representations and do not fundamentally affect OrionLink's design. That said, the privacy guarantees targeted by OrionLink are not required in all SSO scenarios. In enterprise settings using SAML 2.0 or WS-Federation, RPs and IdPs are often under common organizational control and already have visibility into users' identities and access patterns, making anonymity less critical. In such environments, anonymous credentials may be unnecessary, while OrionLink's end-to-end encryption can still provide value by protecting authentication data from broker observation.

TCA Security. In OrionLink's deployment, the TCA runs in a cross-origin `<iframe>` on an origin independent of the IdP,

TABLE IV: Comparison of *OrionLink* with prior privacy-preserving SSO schemes.

Scheme	Privacy			Brokered SSO Features		Extension	
	Identity Obliviousness	Credential Obliviousness	Personal Info. Obliviousness	End-to-End Confidentiality	Broker Compatibility	Authorization Mgmt.	Forward-Secure Token Refresh
UP-SSO [46]	✓	✗	✗	✗	✗	✗	✗
BISON [21]	✓	✗	✗	✗	✗	✗	✗
AIF-ZKP [47]	◑ [1]	✗	✗	✗	✗	◑ [2]	✗
ARPSSO [20]	✓	✗	✗	✗	✗	✗	✗
MISO [22]	✓	✓	✓	✗	✗	✗	✗
OrionLink	✓	✓	✓	✓	✓	✓	✓

[1]Unlinkability limited to the User–RP relationship. [2]RP authorization control only indirectly, through validity-period enforcement.

the broker, and any RP, so the same-origin policy prevents IdP-hosted scripts from reading TCA-internal state such as the PIN, K_m, or plaintext D. The TCA page loads its open, version-pinned script bundle via an SRI-pinned `<script>` element, while versioned URLs and an independently auditable transparency log provide bundle provenance. We defer the remaining TCA security details to Appendix E.

Web Attacks and Side Channels. OrionLink builds an end-to-end encrypted channel between the RP and the IdP through AAKA, which resists the network attacks that arise once the broker is inserted as an additional relay point. In standard brokered SSO, a mis-validated redirection chain delivers plaintext authentication tokens to an attacker-chosen endpoint (Appendix B-B), whereas in OrionLink the same attack yields only info_{Enc} and content-free pseudonyms that reveal nothing without the session key K_S shared by the RP and the IdP. The brokered architecture also removes one class of side channels against the RP, since every login request reaches the IdP from the broker, so the IdP observes neither the request fingerprint nor the network traffic of the RP. Side channels beyond the model remain, including the timing of TCA computations and correlation by a global passive adversary, and established countermeasures apply here as well, such as building the TCA from constant-time cryptographic components. A compromised browser or malicious extension instead breaks the client-side trust anchor altogether, and we discuss this case in Appendix E.

VII. Related Works

We review prior work on privacy-preserving SSO and position *OrionLink* within research on brokered authentication.

Privacy-preserving SSO. To the best of our knowledge, *OrionLink* is the first privacy-enhancing framework designed for *brokered* SSO. In contrast, most existing approaches [21]–[23], [33], [49] focus on standard SSO without an intermediary broker. Table IV summarizes a detailed comparison. Several schemes [21]–[23], [34], [46] employ anonymous credentials to achieve properties such as *user behavior untraceability* and *cross-RP unlinkability*, which *OrionLink* also provides. However, many rely on outdated or insecure protocol designs. In particular, multiple proposals [33], [47], [49] are based on the deprecated OIDC Implicit Flow, which lacks RP authentication and is vulnerable to token leakage. By contrast, ARPSSO [20] was the first system built entirely on the modern OIDC Authorization Code flow, a secure-by-design approach that *OrionLink* also adopts. *OrionLink* uniquely supports key brokered SSO properties, including *(i) end-to-end confidentiality*, preventing brokers from accessing tokens or sensitive user data, and *(ii) broker compatibility*, largely preserving the assistive functions critical to existing brokered SSO ecosystems. Although MISO [22] was designed for standard SSO, its TEE-based mixer intermediates communication between the RP and IdP to hide credentials and personal data, conceptually resembling a broker. However, reliance on TEEs introduces additional trust assumptions and deployment complexity. In AIF-ZKP [47], RP authorization is enforced by limiting the lifetime of RP credentials used in anonymous authentication. This approach remains passive and does not support active, user-driven authorization management.

Brokered SSO in Emerging Architectures. The importance of secure brokered SSO has increased with emerging architectures such as MCP, an open standard for connecting AI systems with external applications [14]–[16]. In MCP, servers simultaneously act as IdPs for clients and as RPs for upstream services, naturally fitting the brokered SSO model and making delegated authorization essential. Cloudflare [16], a widely deployed reverse proxy, promotes brokered SSO as a practical means of integrating authorization into MCP, recommending "*authorization-as-a-service*" providers such as Stytch [50], Auth0 [51], and WorkOS [52]. As brokered SSO adoption grows in AI-driven ecosystems, privacy and data exposure risks increase accordingly, highlighting the need for systems like *OrionLink*.

VIII. Conclusion

In this work, we identify data exposure at identity brokers as a fundamental security and privacy risk in brokered SSO and propose **OrionLink**, the first privacy-preserving framework designed for *brokered SSO* to mitigate risks posed by *identity brokers*. OrionLink adopts *broker obliviousness* as its central design principle and minimizes the information observable by the broker throughout protocol execution. Specifically, OrionLink introduces (*a*) an anonymous pseudonym derivation scheme that generates unlinkable, RP-specific user pseudonyms and session-bound RP pseudonyms, preventing cross-service behavioral tracking, and (*b*) an anonymous key agreement protocol that establishes end-to-end encrypted channels between RPs and IdPs, shielding users' authentica-

tion and authorization data from broker visibility. We further instantiate OrionLink in a practical system and incorporate two features to enhance usability and security.

We formally prove the security of OrionLink, and our prototype implementation and experimental evaluation demonstrate that the system achieves all stated design goals with minimal performance overhead, while providing strong privacy guarantees and a seamless user experience. In particular, OrionLink attains an end-to-end login latency of about 2.1 seconds in a WAN setting, underscoring its practicality for deployment in real-world brokered SSO ecosystems.

A. Ethical Considerations

This work presents a defensive, privacy-enhancing framework that does not involve human subjects, real user data, or attacks on deployed systems.

Acknowledgment

This work was supported in part by the Beijing Natural Science Foundation under Grant 4242023; in part by the National Natural Science Foundation of China under Grant 62472021; and in part by the Fundamental Research Funds for the Central Universities.

References

[1] N. Sakimura, J. Bradley *et al.*, "Openid connect core 1.0 incorporating errata set 1," *The OpenID Foundation, specification*, 2014.

[2] D. Hardt, "The oauth 2.0 authorization framework," 2012.

[3] V. L. Pochat, T. Van Goethem, Tajalizadehkhoob *et al.*, "Tranco: A research-oriented top sites ranking hardened against manipulation," *arXiv preprint arXiv:1806.01156*, 2018.

[4] L. Jannett, Westers *et al.*, "Sok: Sso-monitor-the current state and future research directions in single sign-on security measurements," in *2024 IEEE 9th European Symposium on Security and Privacy (EuroS&P)*, 2024.

[5] M. Ghasemisharif, C. Kanich, Polakis *et al.*, "Towards automated auditing for account and session management flaws in single sign-on deployments," in *2022 IEEE Symposium on Security and Privacy (SP)*, 2022.

[6] D. Fett, R. Küsters, and G. Schmitz, "A comprehensive formal security analysis of oauth 2.0," in *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, 2016.

[7] M. Ghasemisharif, A. Ramesh *et al.*, "O single {Sign-Off}, where art thou? an empirical analysis of single {Sign-On} account hijacking and session management on the web," in *27th USENIX security symposium*, 2018.

[8] D. Fett, R. Küsters, and G. Schmitz, "An expressive model for the web infrastructure: Definition and application to the browser id sso system," in *2014 IEEE Symposium on Security and Privacy*, 2014.

[9] G. Liu, X. Gao *et al.*, "An investigation of identity-account inconsistency in single sign-on," in *Proceedings of the Web Conference 2021*, 2021.

[10] P. Philippaerts, D. Preuveneers, and W. Joosen, "Oauch: Exploring security compliance in the oauth 2.0 ecosystem," in *Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses*, 2022.

[11] D. Hardt, A. Parecki, and T. Lodderstedt, "The OAuth 2.1 Authorization Framework," Internet Engineering Task Force, Tech. Rep., 2025. [Online]. Available: https://datatracker.ietf.org/doc/draft-ietf-oauth-v2-1/13/

[12] T. Lodderstedt, J. Bradley *et al.*, "Rfc 9700: Best current practice for oauth 2.0 security," 2025.

[13] T. Innocenti, L. Jannett *et al.*, ""only as strong as the weakest link": On the security of brokered single sign-on on the web," in *IEEE S&P*, 2025.

[14] X. Hou, Y. Zhao, S. Wang *et al.*, "Model context protocol (mcp): Landscape, security threats, and future research directions," *arXiv preprint arXiv:2503.23278*, 2025.

[15] Anthropic. (2025) Understanding mcp servers. [Online]. Available: https://modelcontextprotocol.io/docs/learn/server-concepts

[16] Cloudflare, "Connect, protect, and build everywhere," 2024. [Online]. Available: https://developers.cloudflare.com/

[17] R. Goswami. (2023) Okta hackers stole data on all customer support users in major breach. [Online]. Available: https://www.cnbc.com/2023/11/29/okta-hackers-stole-data-on-all-customer-support-users-company-says.html

[18] 1Password, "Password management," 2025. [Online]. Available: https://1password.com/

[19] S. Casacuberta, J. Hesse *et al.*, "Sok: oblivious pseudorandom functions," in *2022 IEEE 7th European Symposium on Security and Privacy*, 2022.

[20] J. He, L. Lei, Y. Wang *et al.*, "Arpsso: An oidc-compatible privacy-preserving sso scheme based on rp anonymization," in *European Symposium on Research in Computer Security*, 2024.

[21] J. Heher, S. More, and L. Heimberger, "Bison: Blind identification with stateless scoped pseudonyms," in *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, 2025.

[22] R. Xu, S. Yang, F. Zhang *et al.*, "Miso: legacy-compatible privacy-preserving single sign-on using trusted execution environments," in *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*, 2023.

[23] M. Kroschewski *et al.*, "Oppid: single sign-on with oblivious pairwise pseudonyms," *Proceedings on Privacy Enhancing Technologies*, 2025.

[24] A. Davidson, A. Faz-Hernandez, N. Sullivan, and C. Wood, "Rfc 9497: Oblivious pseudorandom functions (oprfs) using prime-order groups," 2023.

[25] A. Davidson, A. Faz-Hernandez, N. Sullivan *et al.*, "Oblivious pseudorandom functions (oprfs) using prime-order groups," Internet-Draft draft-sullivan-cfrg-voprf-03, Internet Engineering Task Force, Tech. Rep., 2019.

[26] S. Jarecki, A. Kiayias, and H. Krawczyk, "Round-optimal password-protected secret sharing and t-pake in the password-only model," in *ASIACRYPT*, 2014.

[27] I. Damgård, "On σ-protocols," *Lecture Notes, University of Aarhus, Department for Computer Science*, 2002.

[28] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Conference on the theory and application of cryptographic techniques*, 1986.

[29] S. A. Kakvi, K. M. Martin, C. Putman *et al.*, "Sok: anonymous credentials," in *International Conference on Research in Security Standardisation*, 2023.

[30] D. Pointcheval and O. Sanders, "Short randomizable signatures," in *Cryptographers' Track at the RSA Conference*, 2016.

[31] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: The second-generation onion router," 2004.

[32] J. Hughes and E. Maler, "Security assertion markup language (saml) v2. 0 technical overview," *OASIS SSTC Working Draft sstc-saml-tech-overview-2.0-draft-08*, 2005.

[33] D. Fett, R. Küsters, and G. Schmitz, "Spresso: A secure, privacy-respecting single sign-on system for the web," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 2015.

[34] J. Lin, B. Zhang, W. Wang *et al.*, "Understanding the identity-transformation approach in oidc-compatible privacy-preserving sso services," *arXiv preprint arXiv:2506.01325*, 2025.

[35] M. Marlinspike and T. Perrin, "The x3dh key agreement protocol," *Open Whisper Systems*, 2016.

[36] T. Perrin and M. Marlinspike, "The double ratchet algorithm," *GitHub wiki*, 2016.

[37] D. Fett, R. Küsters, and G. Schmitz, "The web sso standard openid connect: In-depth formal security analysis and security guidelines," in *2017 IEEE 30th Computer Security Foundations Symposium (CSF)*, 2017.

[38] S. Bowe. (2017) Bls12-381: new zk-snark elliptic curve construction. [Online]. Available: https://electriccoin.co/blog/new-snark-curve/

[39] M. Jones, J. Bradley, and N. Sakimura, "Json web token (jwt)," Internet Engineering Task Force, Tech. Rep. RFC 7519, 2015.

[40] A. Faz-Hernandez and K. Kwiatkowski, *Introducing CIRCL: An Advanced Cryptographic Library*, Cloudflare, 2019. [Online]. Available: https://github.com/cloudflare/circl

[41] P. Miller, *Noble-curves*, 2019. [Online]. Available: https://github.com/paulmillr/noble-curves
[42] V. Nikolaenko and K. Lewi., *Ristretto255*, 2023. [Online]. Available: https://github.com/facebook/ristretto255-js
[43] H. de Valence, J. Grigg, M. Hamburg *et al.*, "The ristretto255 and decaf448 Groups," RFC 9496, 2023.
[44] Microsoft, "Playwright: Fast and reliable end-to-end testing for modern web apps," https://playwright.dev/, 2020, accessed: 2025-01.
[45] Z. Zhang and M. Król, "El passo: privacy-preserving, asynchronous single sign-on," *arXiv preprint arXiv:2002.10289*, 2020.
[46] C. Guo, F. Lang, Q. Wang *et al.*, "Up-sso: enhancing the user privacy of sso by integrating ppid and sgx," in *2021 International Conference on Advanced Computing and Endogenous Security*, 2022.
[47] M. Kroschewski and A. Lehmann, "Save the implicit flow? enabling privacy-preserving rp authentication in openid connect," *Proceedings on Privacy Enhancing Technologies*, 2023.
[48] O. Standard, "Web services federation language (ws-federation) version 1.2."
[49] S. Hammann, R. Sasse, and D. Basin, "Privacy-preserving openid connect," in *Proceedings of the 15th ACM Asia CCS*, 2020.
[50] Stytch. (2025) A better way to build auth. [Online]. Available: https://stytch.com/
[51] Auth0. (2025) Secure ai agents, humans, and whatever comes next. [Online]. Available: https://auth0.com/
[52] WorkOS. (2025) Your app, enterprise ready. [Online]. Available: https://workos.com/
[53] U. Redirects and Forwards. Owasp cheat sheet series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html
[54] S. Jarecki, H. Krawczyk, and J. Xu, "Opaque: an asymmetric pake protocol secure against pre-computation attacks," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2018, pp. 456–486.
[55] E. Kret and R. Schmidt, "The pqxdh key agreement protocol," *Signal*, 2023.
[56] J. Bootle, V. Lyubashevsky, N. K. Nguyen, and A. Sorniotti, "A framework for practical anonymous credentials from lattices," in *Annual International Cryptology Conference*. Springer, 2023, pp. 384–417.
[57] M. R. Albrecht, A. Davidson, A. Deo, and N. P. Smart, "Round-optimal verifiable oblivious pseudorandom functions from ideal lattices," in *IACR International Conference on Public-Key Cryptography*. Springer, 2021, pp. 261–289.
[58] D. Boneh, "The decision diffie-hellman problem," in *International algorithmic number theory symposium*. Springer, 1998.

APPENDIX A
OPEN SCIENCE

To support transparency and reproducibility, we provide a prototype implementation of our system. The source code is publicly available at https://github.com/Ooo0ption/Orionlink.git.

Our prototype system fully implements all functionalities of the RP, IdP, and broker as designed in OrionLink. The core artifacts included are described in Section V-A.

APPENDIX B
BROKERED SSO ECOSYSTEM

We briefly summarize the current Brokered SSO ecosystem, with key measurements primarily referenced from a recent study [13]. More details and results can be found in the original paper.

A. Measurements of Ecosystem.

Based on publicly available data, the study [13] filtered 55,083 SSO flows considered valid for analysis. As shown in Table V, 249 Brokers were identified in 15,213 SSO flows, with approximately 25% of websites integrating Brokered SSO for authentication.

TABLE V: The Brokered SSO Ecosystem

Model		Brokers		Flows		Websites	
Three-Party		-	-	**39,870**	**72%**	**24,880**	**75%**
Brokered	All Types	**249**	**100%**	**15,213**	**28%**	**8,241**	**25%**
	# Public	64	26%	10,911	71%	5,944	72%
	# Internal	117	47%	3,442	22%	1,635	20%
	# Unknown	68	27%	1,139	7%	686	8%
Total				55,083	100%	33,121	100%

TABLE VI: Vulnerabilities in Brokered SSO Ecosystem

Vulnerabilities	Impact	Brokers	Affected RPs
Open Redirection Chains	Data Leakage	38	1,979
	Account Takeover	34	1,950
	Total	**49**	**2,421**
Unauthorized Data Access	Data Leakage	15	1,103
	Account Takeover	6	779
	Total	**18**	**1,149**

Public vs. Internal Brokers. Public brokers are open to any RP and may be offered as Software as a Service (SaaS) solutions or as open-source options for self-hosted deployments. By contrast, internal brokers serve only a specific set of RPs, often within the same organization such as government agencies.

As shown in Table V, public brokers have a broader impact on the overall brokered SSO ecosystem. We therefore focus more on the privacy and vulnerability issues introduced by public brokers.

B. Measurements of Vulnerabilities.

The prior work [13] identified three major vulnerabilities currently faced by brokered SSO:

- *Open redirection chains [53].* About 20% (49 of 249) of brokers fail to sufficiently validate redirection chains, leading to the leakage of authentication tokens and, in turn, enabling account takeover attacks. (see Table VI)
- *Unauthorized Data Access.* Some brokers lack strict access control and authorization validation mechanisms, allowing attackers to access user data without authorization. This results in data leakage and account overtake attacks. (see Table VI)
- *Violations of SBCPs.* Some brokers (with 3,367 violations observed out of 5,668 flows) fail to adhere to the recommended specifications, thereby weakening the security guarantees of SSO flows.

APPENDIX C
OIDC TOKENS

OIDC issues three main types of tokens to enable authentication and authorization [1]:

- *ID Token*: A signed JSON Web Token issued by the IdP after user authentication. It conveys user identity claims and

includes fields such as `iss` for the issuer, `sub` for the user identifier, and `aud` for the intended audience or client.

- *Access Token*: A token issued by the IdP that allows the client to access protected resources or APIs on behalf of the user. It carries authorization information such as scopes and resource identifiers but does not expose full user identity details.
- *Refresh Token*: A long-lived token issued together with the access token. It enables the client to obtain new access tokens without requiring the user to re-authenticate.

APPENDIX D
MORE DETAILS OF *OrionLink*

Notations. The notations used in the Token Refresh Protocol are defined as follows. (i) RK, the root key, initialized from the session key K_S established via the AAKA protocol. The ratchet state $(\mathsf{RK}, \mathsf{CK})$ is held only by the RP and the IdP, the two parties that know K_S, while the broker keeps no key state and only relays headers and ciphertexts. (ii) CK, the chain key, used by the IdP as a sending chain to derive symmetric message keys. (iii) $\mathsf{KDF_RK}(\mathsf{RK}, dh)$, a KDF that updates the root key and produces a new chain key from the current root key and a fresh DH shared secret dh. (iv) $\mathsf{KDF_CK}(\mathsf{CK})$, a KDF that derives the next chain key and a message key MK from the current chain key.

AAKA Protocol AnonyAKA This protocol is described in Sections IV-D3 and IV-D5. Figure 9 shows the Anonymous Authentication Key Agreement (AAKA) protocol, which is an end-to-end session key establishment protocol that allows the IdP to verify RP legitimacy under anonymity and derive a unique session key for each authorization process.

Proof Generation and Verification. The algorithms are invoked in the protocol flow described in Section IV-D5. Figure 10 shows (1) GenProof, that produces a randomized signature $\bar{\sigma}'$ from $\mathcal{SC}$, an ephemeral public identity key rpk'_{IK} for the AAKA protocol, and a NIZK proof π_2; (2)VerifyProof, that allows the IdP to check the validity of π_2. The NIZK proof π_2 is obtained by applying the Fiat–Shamir transform to a standard Σ-protocol, and its zero-knowledge and soundness properties follow directly in the random oracle model.

Token Refresh Protocol RefToken**.** This protocol is described in Section IV-E2. Figure 11 shows a token refresh protocol that employs a double-ratchet–based key rotation mechanism to protect token confidentiality from broker surveillance.

AnonyAKA between RP and IdP via Broker

Input:

- RP holds the proof generation materials $\mathcal{SC} = (D, \mathsf{rsk}_{IK}, \bar{\sigma})$,$acid$, and the IdP's credential public key ivk_c, identity key ipk_{IK};
- IdP holds its identity key pair $(\mathsf{isk}_{IK}, \mathsf{ipk}_{IK})$ and the credential key pair $(\mathsf{isk}_c, \mathsf{ivk}_c)$.

Output: session key K_S for RP and IdP.

Initialization: IdP pre-generates a signed prekey pair $(\mathsf{isk}_E, \mathsf{ipk}_E)$ and publishes ipk_E together with its signature $\mathsf{Sig}_{\mathsf{isk}_{IK}}(\mathsf{ipk}_E)$ via a public endpoint. The pair is refreshed once per epoch, and isk_E is erased on rotation.

RP: Executes the following steps.

1) generates proof
$$(\pi_2, \bar{\sigma}', (\mathsf{rsk}'_{IK}, \mathsf{rpk}'_{IK})) \leftarrow \mathsf{GenProof}(\mathcal{SC}, acid, \mathsf{ivk}_c)$$
2) requests the ipk_E and its signature and verifies the signature by ipk_{IK}.
3) generates an ephemeral keypair $(\mathsf{rsk}_E, \mathsf{rpk}_E)$.
4) derives the shared session key K_S with the initiator routine
$$K_S \leftarrow \mathsf{X3DH}_{\mathsf{RP}}(\mathsf{rsk}'_{IK}, \mathsf{rsk}_E, \mathsf{ipk}_{IK}, \mathsf{ipk}_E)$$
5) constructs the associated data $\mathsf{AD} = (\mathsf{rpk}'_{IK} || \mathsf{ipk}_{IK})$ and prepares the initial message:
$$\mathsf{msg}_{init} = \mathsf{AEAD.Enc}_{K_S}(\mathsf{rpk}_E || \mathsf{ipk}_E, \mathsf{AD})$$
6) set $M = (\pi_2, \bar{\sigma}', \mathsf{rpk}'_{IK}, \mathsf{rpk}_E, \mathsf{msg}_{init})$.

IdP: On receiving $acid, M = (\pi_2, \bar{\sigma}', \mathsf{rpk}'_{IK}, \mathsf{rpk}_E, \mathsf{msg}_{init})$ from RP

1) sets $\sigma' = acid$, verifies the proof π_2
$$(0/1) \leftarrow \mathsf{VerifyProof}(\pi_2, \sigma', \bar{\sigma}', \mathsf{rpk}'_{IK}, (\mathsf{isk}_c, \mathsf{ivk}_c))$$
2) derives the shared session key K_S with the mirrored responder routine
$$K_S \leftarrow \mathsf{X3DH}_{\mathsf{IdP}}(\mathsf{isk}_{IK}, \mathsf{isk}_E, \mathsf{rpk}'_{IK}, \mathsf{rpk}_E)$$
3) reconstructs the associated data $\mathsf{AD} = (\mathsf{rpk}'_{IK} || \mathsf{ipk}_{IK})$ and decrypt $\mathsf{AEAD.Dec}_{K_S}(\mathsf{msg}_{init}, \mathsf{AD})$.
4) If msg_{init} decrypts successfully, the IdP accepts the K_S; else aborts.

X3DH Routines of AnonyAKA

Inputs: the identity keys of both parties, the RP's ephemeral key, and the IdP's signed prekey.

Output: shared session key K_S.

$\mathsf{X3DH}_{\mathsf{RP}}(\mathsf{rsk}'_{IK}, \mathsf{rsk}_E, \mathsf{ipk}_{IK}, \mathsf{ipk}_E) \rightarrow K_S$, run by the RP as initiator:

$$dh_1 = \mathsf{DH}(\mathsf{rsk}'_{IK}, \mathsf{ipk}_E), \quad dh_2 = \mathsf{DH}(\mathsf{rsk}_E, \mathsf{ipk}_{IK})$$
$$dh_3 = \mathsf{DH}(\mathsf{rsk}_E, \mathsf{ipk}_E), \quad K_S = \mathsf{KDF}(dh_1 || dh_2 || dh_3)$$

$\mathsf{X3DH}_{\mathsf{IdP}}(\mathsf{isk}_{IK}, \mathsf{isk}_E, \mathsf{rpk}'_{IK}, \mathsf{rpk}_E) \rightarrow K_S$, run by the IdP as responder:

$$dh_1 = \mathsf{DH}(\mathsf{isk}_E, \mathsf{rpk}'_{IK}), \quad dh_2 = \mathsf{DH}(\mathsf{isk}_{IK}, \mathsf{rpk}_E)$$
$$dh_3 = \mathsf{DH}(\mathsf{isk}_E, \mathsf{rpk}_E), \quad K_S = \mathsf{KDF}(dh_1 || dh_2 || dh_3)$$

Fig. 9: Anonymous Authentication Key Agreement.

APPENDIX E
TRUSTED COMPUTATION AGENT

The Trusted Computation Agent (TCA) is the sole client-side trusted compute unit of OrionLink, instantiated inside the user's browser as a cross-origin `<iframe>`. It executes user-side cryptographic operations on the user's behalf, removing the cryptographic burden from the RP client and avoiding any user-side deployment such as a browser extension. As the sole *client-side trust anchor* of OrionLink, the TCA must satisfy two security goals.

- *Trusted execution of cryptographic operations.* The TCA must faithfully execute the prescribed user-side cryptographic operations, in particular AC.VerifyCred for consent-time RP verification and the OPRF subprotocol for PIN-derived key generation, so that the D rendered on the consent page is exactly the value the IdP signed into $acid$ and the management key K_m is derived from a genuine OPRF evaluation rather than a malicious substitute.
- *Confidentiality of process data.* Intermediate values produced or held during TCA execution, including the PIN,

$(\pi_2, \bar{\sigma}', (\mathsf{rsk}'_{IK}, \mathsf{rpk}'_{IK})) \leftarrow \underline{\mathsf{GenProof}(\mathcal{SC}, acid, \mathsf{ivk}_c)}$

RP inputs:

- the secret credential $\mathcal{SC} = (D, \mathsf{rsk}_{IK}, \bar{\sigma})$.
- $acid = \sigma' = (\sigma'_1, \sigma'_2)$ with randomness k_1, t_1
- credential verification key ivk_c.

Output: a NIZK proof $\pi_2 = (\vec{C}, \vec{R}, \vec{z})$, an ephemeral identity keypair $(\mathsf{rsk}'_{IK}, \mathsf{rpk}'_{IK})$, randomized signature $\bar{\sigma}'$.

RP executes the following steps:

1) Randomize signature $\bar{\sigma}$ in $\mathcal{SC}$:

$$k_2, t_2 \xleftarrow{\$} \mathbb{Z}_p^*, \bar{\sigma}' = (\bar{\sigma}_1^{'}, \bar{\sigma}_2^{'}) = (\bar{\sigma}_1^{k_2}, (\bar{\sigma}_2 \bar{\sigma}_1^{t_2})^{k_2})$$

2) Generate an ephemeral public identity key:

$$t_3 \xleftarrow{\$} \mathbb{Z}_p^*, \mathsf{rsk}'_{IK} = t_3 + \mathsf{rsk}_{IK}, \mathsf{rpk}'_{IK} = g_1^{t_3 + \mathsf{rsk}_{IK}}$$

3) Sample randomness: $r_1, r_2, r_3, r_D, r_{rsk} \xleftarrow{\$} \mathbb{Z}_p^*$
4) Compute commitment $\vec{C} = (C_1, C_2, C_3)$ and response commitment $\vec{R} = (R_1, R_2, R_3)$

$$C_1 = g_2^{t_1} \cdot \bar{Y}_1^D, \qquad R_1 = g_2^{r_1} \cdot \bar{Y}_1^{r_D}$$
$$C_2 = g_2^{t_2} \cdot \bar{Y}_{21}^{\mathsf{rsk}_{IK}} \cdot \bar{Y}_{22}^D, \qquad R_2 = g_2^{r_2} \cdot \bar{Y}_{21}^{r_{\mathsf{rsk}}} \cdot \bar{Y}_{22}^{r_D}$$
$$C_3 = g_1^{t_3} \cdot g_1^{\mathsf{rsk}_{IK}}, \qquad R_3 = g_1^{r_3} \cdot g_1^{r_{\mathsf{rsk}}}$$

5) Compute challenge $c = H(C_1||C_2||C_3||R_1||R_2||R_3)$
6) Compute response vector $\vec{z} = (z_1, z_2, z_3, z_D, z_{\mathsf{rsk}})$

$$z_1 = r_1 + c \cdot t_1, \quad z_2 = r_2 + c \cdot t_2, \quad z_3 = r_3 + c \cdot t_3$$
$$z_D = r_D + c \cdot D, \quad z_{\mathsf{rsk}} = r_{\mathsf{rsk}} + c \cdot \mathsf{rsk}_{IK}$$

$(0/1) \leftarrow \underline{\mathsf{VerifyProof}(\pi_2, \sigma', \bar{\sigma}', \mathsf{rpk}'_{IK}, (\mathsf{isk}_c, \mathsf{ivk}_c))}$

IdP inputs:

- two signatures $\sigma', \bar{\sigma}'$ and NIZK proof π_2
- credential keypair $(\mathsf{isk}_c, \mathsf{ivk}_c)$ and public key rpk'_{IK}.

Output: accept iff all checks pass; otherwise reject.

IdP executes the following steps:

1) extract $\vec{C}, \vec{R}, \vec{z}$ from π_2;
2) re-compute the challenge $c = H(C_1||C_2||C_3||R_1||R_2||R_3)$
3) check the following equations:

$$C_1^c \cdot R_1 \stackrel{?}{=} g_2^{z_1} \cdot \bar{Y}_1^{z_D}$$
$$C_2^c \cdot R_2 \stackrel{?}{=} g_2^{z_2} \cdot \bar{Y}_{21}^{z_{\mathsf{rsk}}} \cdot \bar{Y}_{22}^{z_D}$$
$$C_3^c \cdot R_3 \stackrel{?}{=} g_1^{z_3} \cdot g_1^{z_{\mathsf{rsk}}}$$

4) check $C_3 = \mathsf{rpk}'_{IK}$ and the two pairing equations:

$$e(\sigma'_1, C_1 \cdot \bar{X}_1) \stackrel{?}{=} e(\sigma'_2, g_2), \quad e(\bar{\sigma}'_1, C_2 \cdot \bar{X}_2) \stackrel{?}{=} e(\bar{\sigma}'_2, g_2)$$

Fig. 10: The Proof Algorithm in AAKA Protocol

RefToken between RP, IdP and Broker

Input:

- RP and IdP hold the session key K_S established from AAKA protocol.
- Broker holds the refresh_token

Output: a new access_token for RP.

Initialization:

- RP sets its initial state:
  - initializes RK $\leftarrow K_S$, CK $\leftarrow \perp$
  - pre-computes a list of DH key pairs $\{(\mathsf{rsk}_i^{dh}, \mathsf{rpk}_i^{dh})\}_{i=0}^{n-1}$
  - sets current DH key state: $\mathrm{DH}_A^{rp} \leftarrow \perp$, $\mathrm{DH}_B^{rp} \leftarrow \perp$
- Broker sets its initial state:
  - initializes $seq \leftarrow 0$
  - loads $\{\mathsf{rpk}_i^{dh}\}_{i=0}^{n-1}$ from RP
  - sets current DH key state: $\mathrm{DH}_A^{idb} \leftarrow \mathsf{rpk}_0^{dh}$, $\mathrm{DH}_B^{idb} \leftarrow \perp$
  - holds no ratchet key state, since it never learns K_S
- IdP sets its initial state:
  - initializes RK $\leftarrow K_S$, CK $\leftarrow \perp$
  - generates DH key pair $(\mathsf{isk}^{dh}, \mathsf{ipk}^{dh})$
  - sets current DH key state $\mathrm{DH}_A^{idp} \leftarrow \perp$, $\mathrm{DH}_B^{idp} \leftarrow \mathsf{ipk}^{dh}$

Broker:

On receiving token request from the RP and the access token is expired:

1) constructs message header $\mathsf{hdr} = \mathrm{DH}_A^{idb}$
2) generates token request message: $\mathsf{msg} = (\mathsf{hdr}, seq, \mathsf{refresh_token})$

On receiving (hdr, seq, c) from the IdP:

1) if $\mathrm{DH}_B^{idb} \neq \mathsf{hdr}$:
   - updates $\mathrm{DH}_B^{idb} \leftarrow \mathsf{hdr}$
   - advances local key: $seq \leftarrow seq + 1$, $\mathrm{DH}_A^{idb} \leftarrow \mathsf{rpk}_{seq}^{dh}$

IdP:

On receiving $\mathsf{msg} = (\mathsf{hdr}, seq, \mathsf{refresh_token})$ from the broker

1) if $\mathrm{DH}_A^{idp} \neq \mathsf{hdr}$:
   - updates $\mathrm{DH}_A^{idp} \leftarrow \mathsf{hdr}$
   - generates new $(\mathsf{isk}^{dh}, \mathsf{ipk}^{dh})$ and updates $\mathrm{DH}_B^{idp} \leftarrow \mathsf{ipk}^{dh}$
   - updates $(\mathrm{RK}, \mathrm{CK}) \leftarrow \mathsf{KDF_RK}(\mathrm{RK}, \mathsf{DH}(\mathsf{isk}^{dh}, \mathrm{DH}_A^{idp}))$
2) derives $\mathrm{CK}, \mathrm{MK} \leftarrow \mathsf{KDF_CK}(\mathrm{CK})$
3) encrypts $c = \mathsf{ENC}_{\mathrm{MK}}(m)$
4) construct message header $\mathsf{hdr} = \mathrm{DH}_B^{idp}$
5) sends (hdr, seq, c) to RP via Broker

RP:

On receiving (hdr, seq, c) from the broker:

1) if $\mathrm{DH}_B^{rp} \neq \mathsf{hdr}$:
   - updates $\mathrm{DH}_B^{rp} \leftarrow \mathsf{hdr}$
   - gets private key $\mathsf{rsk}_{dh} \leftarrow \mathsf{rsk}_{seq}^{dh}$
   - updates $(\mathrm{RK}, \mathrm{CK}) \leftarrow \mathsf{KDF_RK}(\mathrm{RK}, \mathsf{DH}(\mathsf{rsk}_{dh}, \mathrm{DH}_B^{rp}))$
2) derives new message key $\mathrm{CK}, \mathrm{MK} \leftarrow \mathsf{KDF_CK}(\mathrm{CK})$
3) decrypts $m = \mathsf{DEC}_{\mathrm{MK}}(c)$

Fig. 11: Token Refresh with Key Rotation Mechanism

the management key K_m, the OPRF blinding factor, and the plaintext D, must not become observable to the IdP back end, the broker, or any third-party script in the user's browser.

Implementation Details. The TCA is loaded as a cross-origin `<iframe>` embedded in the relevant IdP-hosted pages and performs two functions tied to specific OrionLink phases.

- *Consent-time RP verification* (Section IV-D4). The broker iframe forwards $(cid, acid, \pi_D, D)$ to the TCA via `window.postMessage`. The TCA then parses $\pi_D = t_1$ and evaluates $\mathsf{AC.VerifyCred}(\mathsf{ivk}_c, D, acid, \pi_D)$, which derandomizes $acid$ with t_1 before checking the signature, and, on success, renders D inside its own iframe surface for user confirmation.
- *Private authorization management* (Section IV-E1). The broker iframe delivers $(acid, D_{\mathsf{Enc}})$, the TCA prompts the user for a short PIN, runs the OPRF subprotocol with the IdP to derive $K_m = \mathsf{KDF}(\mathsf{Unblind}(r', \mathsf{BlindEval}(\mathsf{k}_{oprf}, \mathsf{Blind}(\mathsf{PIN}))))$, decrypts $D \leftarrow \mathsf{Dec}_{K_m}(D_{\mathsf{Enc}})$, renders the plaintext D for review or revocation, and emits only $(acid, \text{decision})$ to the broker.

TCA Security. In practice, the TCA's security is achieved not through bespoke cryptographic constructions but through a small set of concrete frontend deployment strategies that modern browsers provide natively. We combine three such standard web-platform primitives to jointly enforce the two goals above.

- *Browser same-origin policy.* Because the TCA, the broker iframe, and the IdP page each live in distinct origins, the browser same-origin policy prevents any IdP-hosted script from reading TCA-internal state. The only cross-frame channel is `window.postMessage`. Each sender pins the recipient with an explicit `targetOrigin`, and the receiving frame accepts a message only after checking `event.origin` and `event.source` against the expected counterpart, validating the message structure, and matching a fresh per-session nonce bound to the session identifier, so that cross-origin injection and replayed messages are rejected.
- *Runtime integrity via Subresource Integrity.* The TCA document loads its external script bundle through an SRI-pinned `<script>` element, so the browser refuses to execute any bundle whose bytes deviate from the pinned hash, binding runtime execution to the exact release named by the versioned bundle URL. Users can also recompute the bundle hash on demand to independently verify its integrity.
- *Auditable and upgradeable provenance.* The TCA bundle is served from a publicly auditable host independent of any IdP, broker, or RP, and the reference hash of every release is recorded in an append-only binary transparency log analogous to Certificate Transparency. This supports controlled version upgrades while making silent divergence between users, as well as retroactive replacement of a past release, globally detectable.

Hardening against a Malicious IdP. The mechanisms above suffice under our honest-but-curious model. Stronger guarantees against an actively malicious IdP can be obtained by redistributing the canonical TCA bundle through out-of-band channels (e.g., a browser extension, a signed WebAssembly module, or RP-operated transparency-log mirrors), so that bundle delivery no longer depends on the IdP-served `<iframe>` tag. A strictly malicious IdP combining a backdoored bundle, a forged TCA host, or browser zero-days falls outside this model and is left to future work, aligning with how prior privacy-preserving SSO systems such as ARPSSO [20] treat equivalent client-side components.

APPENDIX F
ADDITIONAL DISCUSSION

Scope of RP Anonymity. RP anonymity in OrionLink is defined relative to the IdP rather than to the broker. The protected property is the *user–RP authorization relationship*, namely which user has authorized which RP, and it is the IdP from which this relationship stays hidden. The broker needs D and cid in the clear to retrieve the RP configuration, route the request, and bind the IdP response back to the RP session, and the BrokerView outputs in Figure 4 grant this metadata explicitly. Knowing the RP therefore only lets the broker associate it with the per-RP pseudonym uid_{rp}, which carries no user attributes and stays unlinkable across RPs, so Definitions H.2 and H.3 are unaffected.

Discussion of Forward Secrecy. In our X3DH variant the RP contributes session-level secrets, the ephemeral rsk_E and the randomizer t_3, which it erases after each session, while the IdP contributes its long-lived identity key isk_{IK} and a signed prekey isk_E refreshed once per epoch. Losing both IdP secrets exposes one prekey epoch rather than all past sessions, and shortening the epoch interval mitigates this exposure.

OrionLink can be upgraded to session-level forward secrecy at an acceptable additional cost. In step ⑧ of Figure 5 the IdP samples a one-time key pair $(\mathsf{isk}_O, \mathsf{ipk}_O)$, derives $K'_S = \mathsf{KDF}(K_S \| \mathsf{DH}(\mathsf{isk}_O, \mathsf{rpk}_E))$, erases isk_O, and returns ipk_O alongside info_{Enc}. On receiving the response, the RP rebuilds K'_S from rsk_E, and both sides use it in place of K_S. Refreshed tokens already ratchet in fresh secrets and are unaffected.

Hardening Private Authorization Management. The private authorization management of Section IV-E1 is an optional feature that lets a user review and revoke authorizations from any device with a memorized PIN. Because the key is derived through an OPRF in the manner of the aPAKE design OPAQUE [54], no party that lacks the OPRF key k_{oprf} can mount an offline dictionary attack on the PIN, which covers the broker, the RPs, a network attacker, and anyone who obtains the stored D_{Enc}. One limitation remains, since the IdP both holds k_{oprf} and stores D_{Enc} and can therefore enumerate a short PIN offline in principle. A deployment can remove this capability with a $(2, 2)$-distributed OPRF evaluated jointly by the IdP and the broker [26], which turns every guess into a rate-limited online interaction under our non-collusion assumption, or with a key bound to user-side hardware such as a WebAuthn passkey, at some cost in convenience.

Post-Quantum Migration. OrionLink composes standard primitives, so a post-quantum upgrade can proceed primitive by primitive. The X3DH core of AAKA can be replaced by PQXDH [55], which Signal already deploys at scale, and the PS-based credential by a lattice-based anonymous credential scheme [56] exposing the same issuance and presentation interface. The DH-based OPRF that derives uid_{rp} and K_m has lattice-based candidates [57]. A full post-quantum instantiation and its evaluation remain future work.

APPENDIX G
PRELIMINARY

In this section, we describe the security properties required of the building blocks used in our construction. Let λ denote the security parameter, and let $\mathcal{A}$ be a probabilistic polynomial-time (PPT) adversary.

A. Security Properties of OPRFs

Under the Decisional Diffie–Hellman (DDH) problem [58] in the random oracle model, the OPRF used in our construction satisfies correctness, consistency, and pseudorandomness.

1) *Correctness.* For any λ, any secret key k, and any input x, an honest execution of the $\mathsf{Blind}(\cdot)$, $\mathsf{BlindEval}(\cdot)$, and $\mathsf{Unblind}(\cdot)$ algorithms outputs $\mathsf{OPRF}_k(x)$, except with $\mathsf{negl}(\lambda)$.

2) *Consistency.* For any λ, for any $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that

$$\Pr[\mathsf{Exp}^{\mathsf{cons}}_{\mathsf{OPRF}}(\mathcal{A},\lambda)=1] \geq 1-\mathsf{negl}(\lambda),$$

where the experiment $\mathsf{Exp}^{\mathsf{cons}}_{\mathsf{OPRF}}(\mathcal{A},\lambda)$ is defined as follows:

- The challenger samples a secret key $k \xleftarrow{\$} \mathbb{Z}_p$ and an input x.
- The challenger runs two independent honest executions of the OPRF protocol on input x, obtaining outputs y_1 and y_2.
- The experiment outputs 1 iff $y_1 = y_2$.

3) *Pseudorandomness.* For any λ, for any $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that

$$\left|\Pr[\mathsf{Exp}^{\mathsf{pseud}}_{\mathsf{OPRF}}(\mathcal{A},\lambda)=1]-\tfrac{1}{2}\right| \leq \mathsf{negl}(\lambda)$$

define the experiment $\mathsf{Exp}^{\mathsf{pseud}}_{\mathsf{OPRF}}(\mathcal{A},\lambda)$ as follows:

- The challenger samples a secret key $k \xleftarrow{\$} \mathbb{Z}_p$ and keeps it secret.
- $\mathcal{A}$ is given adaptive oracle access to $\mathsf{OPRF}_k(\cdot)$, implemented via the honest $\mathsf{BlindEval}(\cdot)$ and $\mathsf{Unblind}(\cdot)$.
- Eventually, $\mathcal{A}$ outputs a challenge input x' that has not been queried before.
- The challenger samples a random bit $b \xleftarrow{\$} \{0,1\}$. If $b = 0$, it sets $y' := \mathsf{OPRF}_k(x')$; if $b = 1$, it sets $y' \xleftarrow{\$} \mathbb{G}$. The challenger returns y' to $\mathcal{A}$.
- $\mathcal{A}$ outputs a guess b'.
- The experiment outputs 1 iff $b' = b$.

B. Security Properties of ACs

Under the XDH assumption, implying the hardness of the Computational co-Diffie–Hellman (co-CDH) problem in $\mathbb{G}_1$ and $\mathbb{G}_2$ and the Decisional Diffie–Hellman (DDH) problem in $\mathbb{G}_1$ [30], where all zero-knowledge proofs are instantiated using the Fiat–Shamir transform of underlying Σ-protocols. The anonymous credential protocol that we used satisfies correctness, unforgeability, blindness, unlinkability/zero-knowledge.

1) *Correctness.* For any λ, and any attribute vector $A = (A_h, A_p)$ such that $\phi(A_h) = 1$, where ϕ is an efficiently computable predicate on hidden attributes, it holds that

$$\Pr\left[\begin{array}{l}(pp) \leftarrow \mathsf{AC.Setup}(1^\lambda);\\ (sk,vk) \leftarrow \mathsf{AC.KeyGen}(pp,n);\\ (A,\sigma) \leftarrow \mathsf{AC.IssueCred}(pp,sk,A_h,A_p);\\ (A'_h,A'_p,\pi,\phi) \leftarrow \mathsf{AC.PresentCred}(vk,\sigma,A):\\ \mathsf{AC.VerifyCred}(vk,A'_p,\pi,\phi)=1\end{array}\right] = 1.$$

2) *Unforgeability.* For any λ, for any $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that

$$\Pr\Big[\mathsf{Exp}^{\mathsf{uf}}_{\mathsf{AC}}(\mathcal{A},\lambda)=1\Big] \leq \mathsf{negl}(\lambda).$$

where the experiment $\mathsf{Exp}^{\mathsf{uf}}_{\mathsf{AC}}(\mathcal{A},\lambda)$ is defined as follows:

- The challenger runs

$$pp \leftarrow \mathsf{AC.Setup}(1^\lambda), (sk,vk) \leftarrow \mathsf{AC.KeyGen}(pp,n)$$

and gives (pp, vk) to $\mathcal{A}$.
- $\mathcal{A}$ is given oracle access to $\mathsf{AC.IssueCred}(\cdot)$ and $\mathsf{AC.PresentCred}(\cdot)$. The challenger maintains a set $\mathcal{Q}$ containing all pairs (A,π) that arise from honest executions of these oracles.
- Eventually, $\mathcal{A}$ outputs a tuple $(\tilde{A},\tilde{\sigma},\tilde{\pi},\phi)$.
- The experiment outputs 1 if

$$\mathsf{AC.VerifyCred}(\tilde{A},\tilde{\sigma},\tilde{\pi},\phi)=1 \quad \text{and} \quad (\tilde{A},\tilde{\pi}) \notin \mathcal{Q};$$

otherwise, the experiment outputs 0.

3) *Blindness.* For any $\mathcal{A}$ controlling the issuer, there exists a negligible function $\mathsf{negl}(\cdot)$ such that

$$\left|\Pr\Big[\mathsf{Exp}^{\mathsf{blind}}_{\mathsf{AC}}(\mathcal{A},\lambda)=1\Big]-\tfrac{1}{2}\right| \leq \mathsf{negl}(\lambda).$$

where the experiment $\mathsf{Exp}^{\mathsf{blind}}_{\mathsf{AC}}(\mathcal{A},\lambda)$ proceeds as follows:

- $\mathcal{A}$ samples two attribute vectors $A^{(0)} = (A_h^{(0)}, A_p)$ and $A^{(1)} = (A_h^{(1)}, A_p)$ such that $\phi(A_h^{(0)}) = \phi(A_h^{(1)}) = 1$.
- A challenger samples $b \leftarrow \{0,1\}$ and runs

$$(A,\sigma) \leftarrow \mathsf{AC.IssueCred}(pp,sk,A^{(b)}),$$

interacting with $\mathcal{A}$ as the issuer.
- $\mathcal{A}$ outputs a guess b'.
- The experiment outputs 1 iff $b' = b$.

4) *Unlinkability/Zero-Knowledge.* For any $\mathcal{A}$, including one that generates the key pair (sk, vk) itself and therefore controls both credential issuance and verification, there exists a negligible function $\mathsf{negl}(\cdot)$ such that

$$\left|\Pr\Big[\mathsf{Exp}^{\mathsf{unlink}}_{\mathsf{AC}}(\mathcal{A},\lambda)=1\Big]-\tfrac{1}{2}\right| \leq \mathsf{negl}(\lambda).$$

where the experiment $\mathsf{Exp}^{\mathsf{unlink}}_{\mathsf{AC}}(\mathcal{A},\lambda)$ is defined as follows:

- $\mathcal{A}$ generates the key pair (sk, vk) and outputs vk together with a predicate ϕ.
- The challenger samples two attribute vectors $A^{(0)} = (A_h^{(0)}, A_p)$ and $A^{(1)} = (A_h^{(1)}, A_p)$ such that $\phi(A_h^{(0)}) = \phi(A_h^{(1)}) = 1$.
- The challenger obtains two credentials $(A^{(0)}, \sigma^{(0)})$ and $(A^{(1)}, \sigma^{(1)})$ by executing $\mathsf{AC.IssueCred}$ with $\mathcal{A}$ as the issuer.
- A bit $b \leftarrow \{0,1\}$ is sampled.
- If $b = 0$, the challenger runs

$$T_1 \leftarrow \mathsf{AC.PresentCred}(vk,\sigma^{(0)},A^{(0)}),$$
$$T_2 \leftarrow \mathsf{AC.PresentCred}(vk,\sigma^{(0)},A^{(0)}).$$

- If $b = 1$, the challenger runs

$$T_1 \leftarrow \mathsf{AC.PresentCred}(vk,\sigma^{(0)},A^{(0)}),$$
$$T_2 \leftarrow \mathsf{AC.PresentCred}(vk,\sigma^{(1)},A^{(1)}).$$

- The challenger gives (T_1, T_2) to $\mathcal{A}$, which outputs a guess b'.
- The experiment outputs 1 iff $b' = b$.

APPENDIX H
SECURITY PROOF OF *OrionLink*

Recall Theorem IV.1. We now prove that the protocol in Figure 6 securely realizes $\mathcal{F}_{\mathsf{OrionLink}}$. Specifically, we show that for any real-world adversary $\mathcal{A}$, we can construct an ideal world simulator $\mathcal{S}$, such that for all environments $\mathcal{Z}$, the ideal execution with $\mathcal{S}$ is indistinguishable from the real execution with $\mathcal{A}$. Throughout, parties communicate over authenticated secure channels, $\mathcal{F}_{\mathsf{OrionLink}}$ delivers its outputs immediately.

We consider four distinct adversary models based on the trust models:

1) **Malicious RP ($\mathcal{A}_{RP}$):** Controls the RP to violate authentication correctness or user identity unlinkability.
2) **Semi-Honest IdP ($\mathcal{A}_{IdP}$):** Follows the protocol but attempts to trace user behavior (i.e., link users with the RPs they access).
3) **Semi-Honest Broker ($\mathcal{A}_{\mathsf{IdB}}$):** Follows the protocol but attempts to violate broker obliviousness (i.e., learn user sensitive data).
4) **Joint Corruption ($\mathcal{A}_{\mathsf{Joint}}$):** The semi-honest broker pools its view with an arbitrary set of malicious colluding RPs, attempting to link user pseudonyms across RPs.

For the sake of completeness of the threat model, we consider users to be potentially malicious. However, since OrionLink inherits the user management and authentication mechanisms of standard SSO systems, it also inherits their user authentication correctness guarantees.

A. UC Simulation Proof

We prove Theorem IV.1 through the four corruption cases listed above.

Case 1: Malicious RP ($\mathcal{A}_{\mathsf{RP}}$): Ensure that the RP cannot break authentication correctness or link a user's identity across multiple RPs.

Simulator $\mathcal{S}_{\mathsf{RP}}$ runs $\mathcal{A}_{\mathsf{RP}}$ internally, and simulates the honest IdP, broker, and user.

Setup. $\mathcal{S}_{\mathsf{RP}}$ initializes the public parameters pp and samples the IdP's long-term keys exactly as in *Initialization phase* in Figure 6.

Registration. Upon receives (D, π_1) from $\mathcal{A}_{\mathsf{RP}}$:

1) $\mathcal{S}_{RP}$ sends $(\mathsf{RegisterRP}, RP, D)$ to $\mathcal{F}_{\mathsf{OrionLink}}$, receives (cid).
2) $\mathcal{S}_{RP}$ signs as IdP with cid: $\sigma \leftarrow \mathsf{PS.Sign}(\mathsf{isk}_c, D)$ and $\hat{\bar{\sigma}} \leftarrow \mathsf{BlindSign}(\mathsf{isk}_c, D, \pi_1)$, then returns $(cid, \sigma, \hat{\bar{\sigma}})$ to $\mathcal{A}_{\mathsf{RP}}$, and locally records $(cid, \mathcal{DC})$ in the broker state.

Login Request. $\mathcal{S}_{\mathsf{RP}}$ receives $(cid, acid, \pi_D, M)$ from $\mathcal{A}_{\mathsf{RP}}$, where $acid$ is the randomized-signature session pseudonym, $\pi_D = t_1$ is the disclosed derandomizer, and $M = (\pi_2, \bar{\sigma}', \mathsf{rpk}'_{IK}, \mathsf{rpk}_E, \mathsf{msg}_{init})$ carries the anonymous authentication proof and the key-agreement payload.

1) $\mathcal{S}_{RP}$ runs the honest IdP's verification of the presentation, evaluating $\mathsf{VerifyProof}(\pi_2, \sigma', \bar{\sigma}', \mathsf{rpk}'_{IK}, (\mathsf{isk}_c, \mathsf{ivk}_c))$ with $\sigma' = acid$ exactly as in the real protocol, and records the outcome.
2) $\mathcal{S}_{RP}$ sends $(\mathsf{InitLogin}, RP, sid, cid, acid)$ to $\mathcal{F}_{\mathsf{OrionLink}}$. Because the RP is corrupted, $\mathcal{F}_{\mathsf{OrionLink}}$ adopts the supplied $acid$ and issues the query $(\mathsf{Judge}, sid, cid, acid)$, which $\mathcal{S}_{RP}$ answers with the recorded verification outcome, so the ideal session proceeds exactly when the real IdP would accept the presentation. $\mathcal{S}_{RP}$ then relays the remaining messages to the intended receivers.

User Authentication & Consent.

1) $\mathcal{S}_{RP}$ forwards (D, π_D) to the simulated TCA and checks $\mathsf{AC.VerifyCred}(\mathsf{ivk}_c, D, acid, \pi_D)$, aborting if the check fails.
2) $\mathcal{S}_{RP}$ sends $(\mathsf{ShowRP}, RP, sid, cid, D)$ to $\mathcal{F}_{\mathsf{OrionLink}}$.
3) When the user returns $decision \in \{\mathsf{approve}, \mathsf{deny}\}$, $\mathcal{S}_{RP}$ sends $(\mathsf{Authorize}, User, sid, uid, decision)$ to $\mathcal{F}_{\mathsf{OrionLink}}$.

Login Response. If $\mathcal{F}_{\mathsf{OrionLink}}$ outputs $(\mathsf{AuthOK}, sid, uid_{rp}, Info)$, $\mathcal{S}_{\mathsf{RP}}$ proceeds as follows:

1) $\mathcal{S}_{\mathsf{RP}}$ maps the ideal identifier uid_{rp} to $\mathbb{Z}_p$ by applying a hash-to-field function $H_{\mathbb{Z}_p}$, extracts the group element σ'_1 from the RP pseudonymous $acid = (\sigma'_1, \sigma'_2)$, and computes $auid := {\sigma'_1}^{uid_{rp}}$, which is sent to $\mathcal{A}_{\mathsf{RP}}$ so that unblinding yields uid_{rp}.
2) Based on the previously received $(acid, M)$ and the honest Id's keys materials, $\mathcal{S}_{RP}$ derives K_S as the honset IdP does, and encrypts $\mathsf{info}_{Enc} = Enc_{K_S}(\mathsf{info})$.
3) Sends $(auid, \mathsf{info}_{Enc})$ to $\mathcal{A}_{\mathsf{RP}}$.

Upon receiving uid_{rp} from $\mathcal{A}_{\mathsf{RP}}$, the simulated broker records (cid, uid_{rp}).

Otherwise, $\mathcal{S}_{\mathsf{RP}}$ aborts the session and returns a rejection to the RP, simulating a failed credential verification.

Now we argue that the ideal execution with $\mathcal{S}_{\mathsf{RP}}$ is indistinguishable from the real execution with $\mathcal{A}_{\mathsf{RP}}$.

Hybrid $\mathbf{H}_0$ This is the real execution of $\Pi_{\mathsf{OrionLink}}$ with $\mathcal{A}_{\mathsf{RP}}$ controlling the RP.

Hybrid $\mathbf{H}_1$ is the same as H_0, except that in the registration phase $\mathcal{S}_{\mathsf{RP}}$ computes $(\sigma, \hat{\bar{\sigma}})$ using the same signing and blind-signing algorithms as the real IdP and returns $(cid, \sigma, \hat{\bar{\sigma}})$ to $\mathcal{A}_{\mathsf{RP}}$. $\mathcal{S}_{\mathsf{RP}}$ samples all keys and randomness according to the same distributions as the honest IdP, H_0 and H_1 are perfectly indistinguishable.

Hybrid $\mathbf{H}_2$ is the same as H_1, except that during the login request and user-consent phases, $\mathcal{S}_{\mathsf{RP}}$ simply forwards all messages between the parties and the ideal functionality $\mathcal{F}_{\mathsf{OrionLink}}$.

Hence, H_1 and H_2 are perfectly indistinguishable.

Hybrid $\mathbf{H}_3$ is the same as H_2, except that the acceptance condition of a login session is made explicit and aligned between the real world and the ideal functionality. In the real world, the IdP proceeds to generate a login response if and only if

$$1 \leftarrow \mathsf{VerifyProof}(\pi_2, \sigma', \bar{\sigma}', \mathsf{rpk}'_{IK}, (\mathsf{isk}_c, \mathsf{ivk}_c)),$$

otherwise the session aborts. In $\mathcal{F}_{\mathsf{OrionLink}}$, the session is finalized if and only if (i) the user has authorized the session and (ii) the finalize message carries the same $acid$ recorded in $\mathsf{Sess}[sid]$, a binding that $\mathcal{F}_{\mathsf{OrionLink}}$ created itself for an honest

RP and, for the corrupted RP, created only after checking that $\mathsf{Owner}[cid]$ is corrupted and that the adversary replied accept to the Judge query.

Consider the two possible mismatch events.

(1) *Real accepts but Ideal rejects.* Suppose the IdP-side verification in the real world outputs 1 while the ideal session is not finalized. If the user did not authorize, the real world also produces no response. An abort in the Judge step is impossible, because $\mathcal{S}_{\mathsf{RP}}$ answers that query by running exactly the real verification. The remaining case is an abort because $\mathsf{Owner}[cid]$ is honest, meaning that $\mathcal{A}_{\mathsf{RP}}$ produced a presentation $(\sigma', \bar{\sigma}', \pi_2)$ that VerifyProof accepts for a cid registered by an honest RP. We then construct $\mathcal{B}$ that wins $\mathsf{Exp}^{\mathsf{uf}}_{\mathsf{AC}}$ by outputting this tuple, which is accepted without arising from any credential issued to the adversary. By the unforgeability of AC, this event occurs only with negligible probability.

(2) *Ideal accepts but Real rejects.* For the corrupted RP this event is excluded by construction, because $\mathcal{F}_{\mathsf{OrionLink}}$ finalizes only sessions whose presentation $\mathcal{S}_{\mathsf{RP}}$ accepted by running the real verification. For an honest RP the event means that a presentation honestly derived from a valid issuer-generated credential is rejected by VerifyCred, which contradicts the correctness of AC and hence occurs only with negligible probability. Since both mismatch events are ruled out by the unforgeability and the correctness of AC, H_2 and H_3 are computationally indistinguishable.

Hybrid $\mathbf{H}_4$ is the same as H_3, except for the RP-specific pseudonym $auid$ returned to the RP. From the view of $\mathcal{A}_{\mathsf{RP}}$, the real user identity uid is never revealed. The RP only obtains uid_{rp}, which is pseudorandom except for consistency across sessions with the same User–RP pair. $\mathcal{S}_{\mathsf{RP}}$ receives a uniformly random uid_{rp} from $\mathcal{F}_{\mathsf{OrionLink}}$. If two sessions correspond to the same User–RP pair, $\mathcal{S}_{\mathsf{RP}}$ outputs consistent values of $auid$, otherwise $auid$ is generated pseudorandomly according to the OPRF.

Hence, H_3 and H_4 are computationally indistinguishable under the pseudorandomness of the OPRF.

Hybrid $\mathbf{H}_5$ is the same as H_4, except for the derivation of the session key K_S. $\mathcal{S}_{\mathsf{RP}}$ derives the session key K_S by honestly executing X3DH exactly as an honest IdP would do in the real world.

Hence, H_4 and H_5 are perfectly indistinguishable.

Since $H_0 \approx_c H_5$, the real execution with $\mathcal{A}_{\mathsf{RP}}$ is computationally indistinguishable from the ideal execution with $\mathcal{S}_{\mathsf{RP}}$ and $\mathcal{F}_{\mathsf{OrionLink}}$.

Case 2: Semi-Honest IdP ($\mathcal{A}_{\mathsf{IdP}}$): Ensure the IdP cannot identify which RP the user is accessing or link multiple sessions.

Simulator $\mathcal{S}_{\mathsf{IdP}}$. Since the IdP is semi-honest, $\mathcal{S}_{\mathsf{IdP}}$ simulates the honest RP, broker and user, and relays all messages between $\mathcal{A}_{\mathsf{IdP}}$ and $\mathcal{F}_{\mathsf{OrionLink}}$.

Hybrid $\mathbf{H}_0$ This is the real execution of $\Pi_{\mathsf{OrionLink}}$ with $\mathcal{A}_{\mathsf{IdP}}$, This hybrid corresponds to the real execution of $\Pi_{\mathsf{OrionLink}}$ with $\mathcal{A}_{\mathsf{IdP}}$. In a login session, the IdP receives a login request forwarded by the broker, consisting of

$$(acid, M), \quad M = (\bar{\sigma}', \pi_2, \mathsf{rpk}'_{IK}, \mathsf{rpk}_E, \mathsf{msg}_{init}),$$

which are the RP's anonymous credentials.

Hybrid $\mathbf{H}_1$ is the same as H_0, except that the credential materials $(acid, \bar{\sigma}', \pi_2)$ are generated by the simulator $\mathsf{AC.PresentCred}(\cdot)$ and $\mathsf{GenProof}(\cdot)$, without using any RP-specific secret or identity. The simulator produces a tuple $(acid^{\mathsf{sim}}, \bar{\sigma}'^{\mathsf{sim}}, \pi_2^{\mathsf{sim}}, rpk'^{\mathsf{sim}}_{IK})$, such that the IdP's verification procedure accepts and whose distribution is computationally indistinguishable from that of an honestly generated anonymous credential presentation. $\mathcal{A}_{\mathsf{IdP}}$ cannot distinguish H_0 and H_1. Otherwise, we can construct a $\mathcal{B}$ to win the unlinkability experiment $\mathsf{Exp}^{\mathsf{unlink}}_{\mathsf{AC}}$ of the anonymous credential scheme with non-negligible advantage.

Hence, H_0 and H_1 are computationally indistinguishable under the unlinkability of AC.

Hybrid $\mathbf{H}_2$ is the same as H_1, except that the remaining components $(\mathsf{rpk}_E, \mathsf{msg}_{init})$ are generated according to their protocol specification. Since rpk_E is an ephemeral, one-time public key sampled uniformly at random, and msg_{init} is an encryption of values derived solely from such ephemeral randomness, these components are independent of the RP's identity. H_1 and H_2 are perfectly indistinguishable.

Since $H_0 \approx_c H_2$, the real execution with $\mathcal{A}_{\mathsf{IdP}}$ is computationally indistinguishable from the ideal execution with $\mathcal{S}_{\mathsf{IdP}}$ and $\mathcal{F}_{\mathsf{OrionLink}}$.

Case 3: Semi-Honest Broker ($\mathcal{A}_{\mathsf{IdB}}$): Ensure that the view of $\mathcal{A}_{\mathsf{IdB}}$ can be simulated from the leakage $\mathcal{L}_{\mathsf{IdB}}$ alone, as required by Definition H.1. In particular, the broker learns neither the user's real identity, nor the contents of $info$, nor the session key K_S.

Simulator $\mathcal{S}_{\mathsf{IdB}}$. Since the broker is semi-honest, $\mathcal{S}_{\mathsf{IdB}}$ simulates the honest IdP, RP, and user, and relays all messages between $\mathcal{A}_{\mathsf{IdB}}$ and $\mathcal{F}_{\mathsf{OrionLink}}$.

Hybrid $\mathbf{H}_0$ The real world protocol execution.

Hybrid $\mathbf{H}_1$ is the same as H_0, except that every message in the broker's view other than the ciphertext info_{Enc} is produced by $\mathcal{S}_{\mathsf{IdB}}$ from the leakage $\mathcal{L}_{\mathsf{IdB}}$ and the randomness of the simulated honest parties. Upon $(\mathsf{BrokerView}, cid, D)$, $\mathcal{S}_{\mathsf{IdB}}$ runs the registration phase for D on behalf of the simulated IdP, whose keys it holds. Upon $(\mathsf{BrokerView}, sid, cid, acid)$, it runs $\mathsf{AC.PresentCred}(\cdot)$ and $\mathsf{GenProof}(\cdot)$ on behalf of the simulated RP with fresh randomizers (k_1, t_1) and ephemeral keys, obtaining the login request $(cid, acid, \pi_D, M)$ that the broker forwards. Upon $(\mathsf{BrokerView}, sid, uid_{rp}, |info|)$, it computes $auid$ from the leaked uid_{rp} and the blinding factor k_1 of the simulated RP, exactly as $\mathcal{S}_{\mathsf{RP}}$ does in Case 1. It also replays the leaked uid_{rp} as the delivery message from the simulated RP to the broker. Every such message is therefore a function of $\mathcal{L}_{\mathsf{IdB}}$ and randomness sampled with the same distribution as in the real protocol, and none of them depends on the user identity uid or the contents of $info$. Hence, H_0 and H_1 are perfectly indistinguishable.

Hybrid $\mathbf{H}_2$ is the same as H_1, except that the real session key K_S derived via AnonyAKA is replaced with a uniformly random key K_R. This step rests on the key indistinguishability of the X3DH key agreement rather than on any property of the encryption scheme. In AnonyAKA,

$K_S = \mathsf{KDF}(dh_1 \| dh_2 \| dh_3)$ is derived from secret keys held only by the RP and the IdP, and the broker observes only the corresponding public keys. Any distinguisher between H_1 and H_2 therefore yields a PPT adversary $\mathcal{B}$ that breaks DDH in the random oracle model, where $\mathcal{B}$ embeds its challenge into the ephemeral Diffie–Hellman terms and answers KDF queries with the random oracle. Hence, H_1 and H_2 are computationally indistinguishable.

Hybrid $\mathbf{H}_3$ is the same as H_2, except that the ciphertext $\mathsf{info}_{Enc} = \mathsf{Enc}_{K_R}(Info)$ is replaced with $\mathsf{Enc}_{K_R}(0^{|Info|})$. Since K_R is now a uniformly random key that is independent of the broker's view, the IND-CPA security of the symmetric encryption scheme applies, and the broker cannot distinguish the encryption of real data from the encryption of zeros. Note that only $|Info|$ is preserved across this hybrid, which is exactly the leakage that BrokerView grants the broker in Figure 4. Hence, H_2 and H_3 are computationally indistinguishable.

Since $H_0 \approx_c H_3$, the real execution with $\mathcal{A}_{\mathsf{IdB}}$ is computationally indistinguishable from the ideal execution with $\mathcal{S}_{\mathsf{IdB}}$ and $\mathcal{F}_{\mathsf{OrionLink}}$. Moreover, the inputs that $\mathcal{S}_{\mathsf{IdB}}$ receives from $\mathcal{F}_{\mathsf{OrionLink}}$ are exactly the leakage $\mathcal{L}_{\mathsf{IdB}}$, so this case establishes the broker obliviousness stated in Definition H.1.

Case 4: Joint Corruption of the Broker and RPs ($\mathcal{A}_{\mathsf{Joint}}$): Ensure that a semi-honest broker that pools its view with an arbitrary set of malicious colluding RPs still cannot link a user's identity across RPs.

Simulator $\mathcal{S}_{\mathsf{Joint}}$. $\mathcal{S}_{\mathsf{Joint}}$ runs $\mathcal{A}_{\mathsf{Joint}}$ internally and simulates the honest IdP, the honest RPs, and the users. Toward the corrupted RPs it behaves exactly as $\mathcal{S}_{\mathsf{RP}}$ in Case 1, verifying every presentation and answering the Judge queries of $\mathcal{F}_{\mathsf{OrionLink}}$. Toward the broker it behaves exactly as $\mathcal{S}_{\mathsf{IdB}}$ in Case 3, producing the broker-visible messages of honest-RP sessions from the leakage $\mathcal{L}_{\mathsf{IdB}}$ and fresh randomness.

Hybrid $\mathbf{H}_0$ The real world protocol execution.

Hybrid $\mathbf{H}_1$ is the same as H_0, except that every message in the joint view other than the pseudonyms and the ciphertexts info_{Enc} of honest-RP sessions is produced by $\mathcal{S}_{\mathsf{Joint}}$. The broker-side messages are generated from $\mathcal{L}_{\mathsf{IdB}}$ exactly as in Hybrid H_1 of Case 3, and the messages exchanged with the corrupted RPs are generated exactly as in Hybrids H_1 through H_3 of Case 1. Hence H_0 and H_1 are indistinguishable by the arguments of those hybrids.

Hybrid $\mathbf{H}_2$ is the same as H_1, except that in honest-RP sessions the session key is replaced with a uniformly random key and info_{Enc} with an encryption of zeros. The coalition holds no key material of these sessions, so the same reductions to DDH in the random oracle model and to IND-CPA apply, and H_1 and H_2 are computationally indistinguishable.

Hybrid $\mathbf{H}_3$ is the same as H_2, except that every pseudonym the coalition obtains, whether unblinded by a corrupted RP or delivered to the broker in an honest-RP session, is replaced by the uniformly random uid_{rp} that $\mathcal{F}_{\mathsf{OrionLink}}$ samples per user–RP pair, embedded consistently as in Case 1. Distinguishing H_2 from H_3 means distinguishing the family $f_{uid}(cid) = cid^{\,uid}$ from an independent random function on the observed user–RP pairs, and a standard hybrid over these pairs reduces each step to DDH in $\mathbb{G}_1$. The reduction embeds a challenge $(g, g^\alpha, g^\beta, Z)$ by implicitly setting the identifier of the target user to α and the cid of the target RP to g^β, samples every other cid as g^r with a known exponent so that all remaining pseudonyms stay computable, and places Z at the target pair. Hence H_2 and H_3 are computationally indistinguishable.

Since $H_0 \approx_c H_3$, the real execution with $\mathcal{A}_{\mathsf{Joint}}$ is computationally indistinguishable from the ideal execution with $\mathcal{S}_{\mathsf{Joint}}$ and $\mathcal{F}_{\mathsf{OrionLink}}$.

Conclusion. Combining these cases, we have proved that $\Pi_{\mathsf{OrionLink}}$ satisfies the defined security goals.

B. Formal Privacy Definitions and Derivations

We now state the three privacy definitions summarized in Section IV-F and establish each of them under the assumptions of Theorem IV.1.

Definition H.1 (Broker Obliviousness)**.** Let $\mathsf{view}^{\Pi}_{\mathsf{IdB}}$ denote the view of an honest-but-curious broker in an execution of a brokered SSO protocol Π, and let $\mathcal{L}_{\mathsf{IdB}}$ denote the leakage granted by BrokerView in Figure 4, namely (cid, D) at RP registration, $(sid, cid, acid)$ at login initialization, and $(sid, uid_{rp}, |info|)$ at login finalization. Π achieves *broker obliviousness* if there exists a PPT simulator $\mathcal{S}_{\mathsf{IdB}}$ such that $\mathcal{S}_{\mathsf{IdB}}(1^\lambda, \mathcal{L}_{\mathsf{IdB}})$ is computationally indistinguishable from $\mathsf{view}^{\Pi}_{\mathsf{IdB}}$.

Derivation. This definition is exactly the statement established by Case 3 above, so no separate reduction is required.

Definition H.2 (Untraceability)**.** A brokered SSO protocol Π achieves *untraceability* if for any λ and any PPT adversary $\mathcal{A}$ there exists a negligible function $\mathsf{negl}(\cdot)$ such that

$$\mathsf{Adv}^{\mathsf{untr}}_{\mathcal{A}} := \left|\Pr[\mathsf{Exp}^{\mathsf{untr}}_{\Pi}(\mathcal{A}, \lambda) = 1] - \tfrac{1}{2}\right| \le \mathsf{negl}(\lambda),$$

where the experiment $\mathsf{Exp}^{\mathsf{untr}}_{\Pi}(\mathcal{A}, \lambda)$ is defined as follows.

- The challenger runs the honest key generation to obtain the issuance key pair $(\mathsf{isk}_c, \mathsf{ivk}_c)$ and hands both keys to $\mathcal{A}$, who plays the IdP.
- The challenger plays two honest RPs with distinct identity descriptors $D_0 \neq D_1$ chosen by $\mathcal{A}$ and obtains their credentials $(\mathcal{DC}_0, \mathcal{SC}_0)$ and $(\mathcal{DC}_1, \mathcal{SC}_1)$ from $\mathcal{A}$ via the registration protocol.
- On each query $d \in \{0, 1\}$ of $\mathcal{A}$, the challenger returns a fresh login request $(acid, M)$ generated from $(\mathcal{DC}_d, \mathcal{SC}_d)$.
- The challenger samples $b \xleftarrow{\$} \{0, 1\}$ and outputs a challenge request $(acid^*, M^*)$ generated from $(\mathcal{DC}_b, \mathcal{SC}_b)$.
- $\mathcal{A}$ outputs a guess b', and the experiment outputs 1 iff $b' = b$.

Derivation. In the experiment the adversary plays the IdP acting as both the credential issuer and the credential verifier, so issuer–verifier collusion is within scope. The environment corrupts the IdP semi-honestly, generates the issuance keys honestly, and hands them to $\mathcal{A}$, exactly as in the experiment. In the ideal execution the IdP observes only $(\mathsf{LoginReq}, sid, acid)$, where $\mathcal{F}_{\mathsf{OrionLink}}$ samples $acid$ fresh at random for honest RPs independently of D_0 and D_1, so the advantage is zero. The transition to the real execution is bounded by Hybrids H_1

and H_2 of Case 2. By Theorem IV.1, the advantage of any PPT adversary in the real execution is therefore negligible. We in fact cover an IdP that generates $(\mathsf{isk}_c, \mathsf{ivk}_c)$ maliciously rather than honestly, because the unlinkability assumption on AC used above already permits the adversary to generate the key pair itself (Section G-B), although such an IdP lies beyond the honest-but-curious scope of our threat model.

Definition H.3 (Unlinkability)**.** A brokered SSO protocol Π achieves *unlinkability* if for any λ and any PPT adversary $\mathcal{A}$ there exists a negligible function $\mathsf{negl}(\cdot)$ such that

$$\mathsf{Adv}^{\mathsf{unlink}}_{\mathcal{A}} := \left| \Pr[\mathsf{Exp}^{\mathsf{unlink}}_{\Pi}(\mathcal{A}, \lambda) = 1] - \tfrac{1}{2} \right| \leq \mathsf{negl}(\lambda),$$

where the experiment $\mathsf{Exp}^{\mathsf{unlink}}_{\Pi}(\mathcal{A}, \lambda)$ is defined as follows.

- $\mathcal{A}$ controls the broker and an arbitrary set of colluding RPs, while the challenger plays the IdP and the honest users.
- The challenger creates users with secret identifiers $uid_i \xleftarrow{\$} \mathbb{Z}_p^*$ and, on each RP registration of $\mathcal{A}$, returns a fresh identifier $cid \xleftarrow{\$} \mathbb{G}_1^*$.
- $\mathcal{A}$ adaptively queries an oracle $\mathcal{O}_{\mathsf{pseu}}(i, cid)$ that returns the pseudonym cid^{uid_i} observed in a login of user i at the RP identified by cid.
- $\mathcal{A}$ outputs two registered identifiers $cid_1 \neq cid_2$ and two user indices $i_0 \neq i_1$.
- The challenger samples $b \xleftarrow{\$} \{0, 1\}$ and returns the challenge pair $(cid_1^{uid_{i_0}}, cid_2^{uid_{i_b}})$.
- $\mathcal{A}$ outputs a guess b', and the experiment outputs 1 iff $b' = b$ and no query to $\mathcal{O}_{\mathsf{pseu}}$ involved cid_2 with index i_0 or i_1.

Derivation. The adversary in this experiment, a broker that pools its view with colluding RPs, matches exactly the corruption pattern of Case 4, and every pseudonym in this joint view, including those the broker observes in honest-RP sessions, is an output of $\mathcal{O}_{\mathsf{pseu}}$. In the ideal execution $\mathcal{F}_{\mathsf{OrionLink}}$ samples every pseudonym independently and uniformly at random, so the advantage is zero. The transition to the real execution is bounded by Hybrid H_3 of Case 4, which reduces to the pseudorandomness of the DH-based OPRF $f_{uid}(cid) = cid^{\,uid}$ and hence to DDH in $\mathbb{G}_1$. By Theorem IV.1, the advantage of any PPT adversary in the real execution is therefore negligible.

The three derivations above yield the following corollary.

Corollary H.1. *Under the assumptions of Theorem IV.1, OrionLink achieves broker obliviousness, untraceability, and unlinkability in the sense of Definitions H.1, H.2, and H.3, respectively.*

C. Security of Forward-Secure Token Refresh

We provide a brief proof that our forward-secure token refresh mechanism achieves both forward secrecy and post-compromise security.

Definition H.4 (Forward Secrecy and Post-Compromise Security)**.** Suppose $\mathcal{A}$ obtains the key for any round of the session, messages sent before or after that round will not be compromised.

Proof Sketch. We utilize a symmetric-key ratchet to achieve *forward secrecy*. Each new session key used by the IdP to encrypt the refreshed access token is derived from the previous one using a key derivation function. And we utilize a Diffie–Hellman ratchet to achieve post-compromise security. The RP pre-generates a list of ephemeral DH key pairs and shares the public keys with the broker. When a token refresh is required, the broker forwards the next public key to the IdP, who performs a new DH exchange to inject fresh randomness into the session key derivation.

APPENDIX I
ARTIFACT APPENDIX

This appendix describes how to reproduce the results in the main body of the paper and is written to be read on its own. The artifact is a complete prototype of *OrionLink*, a privacy-preserving single sign-on (SSO) system. In brokered SSO a user logs in to a website, the relying party (RP), through an intermediary, the broker, which forwards the login to the identity provider (IdP) that authenticates the user and issues the tokens. In *OrionLink* the broker is *oblivious*: it routes logins without learning the user's cross-RP identity or any personal data. The artifact contains the four deployable roles of the paper's Implementation section (Section V-A): IdP, broker, RP, and the Trusted Computation Agent (TCA), a browser-side script on its own origin that performs the user's client-side cryptography. It implements the four protocol phases (Registration, Login Request, User Authentication and Consent, Login Response), the two extensions (private authorization management, forward-secure token refresh), and one measurement harness per experiment of the Evaluation section (Section V-B); about 9K lines of Go and JavaScript in total. "README §x" below refers to `README.md` at the repository root.

A. Description & Requirements

1) How to access: The artifact repository is available via a persistent DOI: https://doi.org/10.5281/zenodo.22815552, and on GitHub: https://github.com/Ooo0ption/Orionlink. The four roles are also published as one prebuilt Docker image, `walnutd/orionlink-ae:ndss2027-v2`.

2) Hardware dependencies: No special hardware. A commodity x86-64 machine with 8 cores, 16 GB of RAM and about 10 GB of free disk runs every experiment and is the basis for the time estimates below. The paper's results were produced on an AMD Ryzen 9 9950X3D (16 cores) with 96 GB of RAM under Ubuntu 24.04.

3) Software dependencies: Two layers. (i) To deploy the prototype and to run E3 the host needs only Docker $\geq$ 24 with the Compose v2 plugin, on Linux (Ubuntu 24.04 recommended) or on Windows with Docker Desktop in Linux-container mode; every command below has a PowerShell equivalent in the README. The image is self-contained and resolves no package from the network at run time. (ii) E1 and E2 are host-side harnesses that drive the containers from outside, so the host additionally needs Go $\geq$ 1.25 (E1 and the unit tests), Node.js $\geq$ 18 (E2, which installs its own Chromium) and Python $\geq$ 3.8 (E2, standard library only). Install commands for Ubuntu, macOS and Windows, with version checks, are in README §3.2; `requirements.txt` is the authoritative list. Nothing is ever installed inside a container.

B. Artifact Installation & Configuration

README §3.1 gives the whole reproduction, from checkout to the three result sets, as one numbered command sequence; the steps here and the experiments below expand its rows. Every command is typed in the host shell at the repository root.

1) **Pull the image** (README §2.2). Set `ORION_IMAGE=walnutd/orionlink-ae:ndss2027-v2` in the shell and run `docker image pull`; the README shows the expected status line. To build the same image from source instead, online or fully offline, follow README §5 and continue here.
2) **Start the containers** (README §2.3). `docker compose up -d --wait` starts the roles in dependency order and returns once all four report healthy; ports 3000–3003 must be free. The README lists the four addresses and marks the only one to open by hand, the RP at `http://localhost:3002/ssso/`.
3) **Install the host-side tools** (README §3.2): `go mod download`, then `npm ci` and `npx playwright install chromium` in `benchmark/e2/browser`, each once.

The stack runs in one of two profiles, selected by `ORION_PROFILE`. The default, `demo`, is the deployment to click through by hand. `bench` runs the same binary and pages but registers the RP at startup, seeds synthetic users and mounts the load-test endpoints; E2 and E3 require it.

Optionally, the front end of the prototype lets the evaluator walk through the *OrionLink* flow one button at a time (README §4), registering the RP, signing in as the demo user, refreshing the token and revoking the authorization. This walk-through is *not* a prerequisite for any experiment.

C. Major Claims

The major claims of the paper that this artifact substantiates are summarized as follows.

- (C1): *OrionLink*'s privacy mechanisms incur only marginal local computation, and the anonymous authenticated key agreement AnonyAKA (AAKA), which RP and IdP run through the broker, is the most expensive stage on the login path. This claim is substantiated by Experiment (E1), which reproduces Table II.
- (C2): End-to-end browser login remains within interactive bounds and degrades gracefully with network delay, growing approximately linearly with the network round-trip time (RTT) for both *OrionLink* and the standard brokered-SSO baseline. This claim is substantiated by Experiment (E2), which reproduces Table III and Fig. 7.
- (C3): The system stays low-latency under sustained load on its most expensive interface, the AnonyAKA path jointly executed by RP, broker and IdP. This claim is substantiated by Experiment (E3), which reproduces Fig. 8.

D. Evaluation

One experiment per claim. Each is driven by a single runner under `benchmark/` and writes its report to `benchmark/results/`; iteration counts and arrival rates are arguments, so an experiment runs at the paper's setting or at a reduced

one. All commands are typed in the host shell at the repository root, and the evaluator never has to enter a container. The two steps that do run inside one, the k6 load generator of E3 and the netem delay setting, are started from the host by Compose.

Experiment (E1) *Local computation overhead* [Table II] (≤10 compute-min; README §3.3). Supports claim C1. It measures each privacy-enhancing stage in isolation and prints the six rows of Table II.

[Configuration Parameters] The paper's setting is 1,000 runs per stage; the argument defaults to 100. Every stage is measured in-process.

[Preparation and Execution] Needs Go on the host (step 3). The stack must be *down* (`docker compose down`): it would compete for CPU, and the unit tests bind port 3000 themselves.

```
go test ./benchmark/unit/
./benchmark/run-e1.sh 1000
```

[Results] The mean time per operation for the six stages, plus their total. The overhead stays small at every stage, which supports the claim that the privacy mechanisms of *OrionLink* add negligible local computation. Absolute values scale with single-core speed, so a deviation of up to a factor of two is expected on slower hardware.

Experiment (E2) *End-to-end latency* [Table III, Fig. 7] (≤45 compute-min; README §3.4). Supports claim C2. A Playwright browser-automation harness drives the running containers and produces every row of Table III: the *SSO Flow* rows from a full login, the *Authorization management* row from the PIN-based decryption in the TCA, and the *Token refresh* row as an authenticated browser request that takes the live RP → broker → IdP path and back.

[Configuration Parameters] The paper's setting is 100 runs per configuration, at emulated RTTs of 0, 40, 100 and 200 ms.

[Preparation and Execution] Needs Node.js and Python on the host (step 3). Start the stack in the bench profile, then run the harness; repeat at each RTT after applying the per-direction delay with `./netem-compose.sh set-egress <delay>` (README §3.6).

```
ORION_PROFILE=bench docker compose up -d --wait
ORION_PROFILE=bench ./benchmark/run-e2.sh 100
```

The brokered OpenID Connect (OIDC) baseline that Table III compares against is a separate stack with its own image and ports, documented in `benchmark/oidc-baseline/`.

[Results] The runner ends by printing Table III (`benchmark/e2/summarize.py` re-renders it from the reports). Browser timings are noisy, so accept a deviation of roughly 50% on absolute values. The claim to check is that the *OrionLink* total at 40 ms RTT is on the order of two seconds and grows linearly with the RTT.

Experiment (E3) *Stress test of the* AnonyAKA *interface* [Fig. 8] (≤20 compute-min; README §3.5). Supports claim C3. It drives the AAKA work of the three roles under a rising arrival rate and produces the three lines of Fig. 8, one per role.

[Configuration Parameters] The paper's setting is an offered load of 100, 500, 1,000, 1,300 and 1,500 requests per second (RPS), generated by k6 with `constant-arrival-rate`, 60 s per rate, for each of the three roles. By default the runner starts a lightweight configuration, 10 and 200 RPS, that any machine sustains; the paper's ladder is requested with `--rps`.

[Preparation and Execution] Needs only Docker. Start the stack in the bench profile with `ORION_NETEM_DELAY` empty, then start the load generator, a fifth Compose service that carries the pinned k6 binary and the E3 scripts inside the submitted image. The command is typed on the host; Compose creates and removes the container.

```
docker compose --profile benchmark run --rm e3
docker compose --profile benchmark run --rm e3 \
    --rps 100,500,1000,1300,1500
```

The first line is the lightweight default (about eight minutes; `--quick` shortens it to two); the second reproduces the paper's ladder. Anything after `e3` is passed to `benchmark/run-e3.sh`.

[Results] One JSON report per role with, for each rate, the target and achieved rate, the p50/p95/p99 latency, the server-side p95 and the dropped-request count; a rate is a valid data point only when nothing was dropped and the achieved rate matches the target. Plot p95 against the rate to obtain Fig. 8. The result to check is the paper's: at 1,000 RPS the p95 latency stays at about 10 ms for the IdP and below that for RP and broker, i.e., flat across the first three rates. Above 1,000 RPS the IdP line rises steeply as its pairing-bound AAKA work saturates the CPU; on a machine with fewer cores than the paper's this knee moves to a lower rate, which does not affect the claim.

E. Customization

The deployment is configured through `.env.example`: `ORION_IMAGE` selects the image, `ORION_PROFILE` the profile, `ORION_NETEM_DELAY` the egress delay, and `ORION_*_URL` / `ORION_*_PORT` relocate the roles. `benchmark/README.md` documents every remaining parameter and the troubleshooting steps.

F. Notes

The prototype is dominated by public-key cryptography and is bound to the CPU, so every experiment is sensitive to other load on the machine and the tolerances quoted above need not cover every deployment; close other workloads first. In (E3) the load generator shares the cores with the server roles and every request crosses the Docker network stack, so running the roles and k6 directly on the host sustains a higher rate.